



UNIVERSITÀ DEGLI STUDI DI FIRENZE

SCUOLA DI INGEGNERIA - DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

Tesi di Laurea Triennale in Ingegneria Informatica

**SVILUPPO DI UNA PIPELINE DI ELABORAZIONE PER SIMULARE
FLUSSI DI EVENTI NEUROMORFICI E ADDESTRARE RETI
NEURALI AL RICONOSCIMENTO DELLE EMOZIONI TRAMITE
L'ANIMAZIONE DI UMANOIDI**

Candidato

Alberto Pizzi

Relatore

Prof. Pietro Pala

Correlatori

Dott. Gabriele Magrini

Prof. Federico Becattini

Anno Accademico 2024/2025

Indice

1	Introduzione	3
1.1	Motivazioni e contesto	3
1.2	Stato dell'arte	3
1.3	Obiettivi	4
2	Tecnologie utilizzate	5
2.1	Software e servizi	5
2.2	Python e librerie	5
2.3	NVIDIA Audio2Face API	5
2.4	Unreal Engine	6
2.5	Simulatori di eventi neuromorfici	6
2.6	Librerie e frameworks per Machine Learning	6
2.7	Hardware	7
3	Pipeline Workflow	8
3.1	Generazione dei CSV a partire dai file audio	8
3.2	Generazione automatica video MP4 da Unreal Engine	9
3.2.1	Generazione Animation Sequences a partire dai CSV	9
3.2.2	Creazione ed assemblaggio Level Sequence	9
3.2.3	Rendering MRQ	10
3.3	Simulazione flusso di eventi neuromorfici	10
3.3.1	Divisione del video in frame	10
3.3.2	Generazione frame neuromorfici PNG	11
3.4	Training reti neurali	11
3.4.1	Creazione training directory	11
3.4.2	Preparazione del dataset e avvio del training	11
4	Datasets audio	12
4.1	CREMA-D	12
4.2	RAVDESS	13
5	Implementazione	15
5.1	Setup progetto e ambienti	15
5.1.1	Setup ambienti Python e Conda	15
5.1.2	Setup di NVIDIA Audio2Face API	15
5.1.3	Setup di Unreal Engine	15
5.1.4	Setup di Frames2Ev	16
5.2	File audio processing	16
5.3	Chiamate batch ad NVIDIA Audio2Face API	18

5.4	Generazione video animazioni con Python API in Unreal Engine	20
5.4.1	Parsing CSV e mappatura blendshapes	21
5.4.2	Generazione Animation Sequences	23
5.4.3	Gestione Sequencer	25
5.4.4	Rendering	27
5.5	Generazione eventi neuromorfici	28
5.5.1	Scomposizione dei video in frames	29
5.5.2	Generazione eventi neuromorfici dai frames	29
6	Training reti neurali	31
6.1	Reti neurali utilizzate	31
6.2	Setup	31
6.3	Iperparametri	32
6.3.1	CrossEntropyLoss	32
6.3.2	Adam	33
6.3.3	Batch size	33
6.3.4	Learning rate	34
6.3.5	Consecutive Sampling dei frame	34
6.3.6	Considerazioni finali	34
6.4	Creazione struttura directory	35
6.5	Progettazione e implementazione	35
6.6	Gestione dei tensori per le reti neurali	36
6.7	Dataset utilizzato	38
6.8	Inferenza	38
7	Risultati	40
7.1	Risultati generazione video RGB sintetici da Unreal	40
7.2	Risultati generazione event-frames dai video	40
7.3	Risultati training	42
7.3.1	Grafici	42
7.3.2	Tempi	44
7.4	Analisi e discussione	45
7.4.1	Analisi risultati generazione video RGB	45
7.4.2	Analisi risultati generazione event-frames	45
7.4.3	Analisi risultati training	45
8	Conclusioni	50
	Glossario	52
	Bibliografia	53
	Ringraziamenti	53

1. Introduzione

1.1 Motivazioni e contesto

Il riconoscimento delle emozioni a partire da dati visivi rappresenta un problema centrale in diversi ambiti applicativi, tra cui Human-Computer Interaction, sistemi di sicurezza e analisi del comportamento umano. La possibilità di interpretare lo stato emotivo di un individuo consente infatti lo sviluppo di sistemi più adattivi e intelligenti.

Tuttavia, l'analisi delle espressioni facciali attraverso dati video presenta ancora diverse limitazioni, legate principalmente alla difficoltà di modellare in modo efficace la componente dinamica del volto e alla dipendenza da dati acquisiti in condizioni spesso controllate.

In questo contesto, emerge la necessità di esplorare rappresentazioni alternative del dato visivo che possano meglio descrivere la natura temporale delle espressioni facciali e supportare scenari di apprendimento più complessi e realistici.

Per questo motivo, risulta rilevante investigare approcci basati su dati generati e rappresentazioni event-based, in grado di fornire una descrizione più orientata alle dinamiche del movimento rispetto alle tradizionali sequenze di immagini.

Si è scelto di adottare segnali audio come input della pipeline, in quanto rappresentano una modalità efficace per codificare l'informazione emotiva in modo naturale e continuo. A partire da tali segnali, viene utilizzato NVIDIA Audio2Face (capitolo 2.3) per la generazione automatica di animazioni facciali realistiche, che garantiscono coerenza temporale e fedeltà delle espressioni, inclusa la sincronizzazione labiale (lip-sync). Questa scelta consente di controllare direttamente l'evoluzione dell'espressione emotiva del soggetto sintetico, facilitando la creazione di sequenze coerenti e utilizzabili per la successiva generazione di dati video e neuromorfici.

1.2 Stato dell'arte

Le attuali soluzioni per il riconoscimento delle emozioni a partire da dati visivi si basano principalmente su tecniche di deep learning applicate a immagini e video RGB. In questi approcci, i modelli analizzano le espressioni facciali per classificare lo stato emotivo del soggetto, ottenendo buone prestazioni in diversi contesti applicativi. Tuttavia, tali metodologie risultano fortemente dipendenti dalla qualità e dalla variabilità dei dataset disponibili, oltre a essere sensibili a fattori come illuminazione, pose e condizioni di acquisizione [1, 8].

Parallelamente, la visione neuromorfica rappresenta un paradigma alternativo basato su sensori *event-based*, che registrano variazioni di luminosità nel tempo invece di acquisire frame completi. Questa rappresentazione consente di catturare in modo più efficace le dinamiche temporali e i micro-movimenti, risultando particolarmente adatta all'analisi delle espressioni facciali. Studi abbastanza recenti hanno evidenziato come l'utilizzo di dati neuromorfici possa migliorare la capacità di riconoscere micro-espressioni difficilmente osservabili nei video RGB tradizionali [2, 13].

Nonostante questi vantaggi, l'applicazione della visione neuromorfica al riconoscimento delle emozioni è ancora limitata, principalmente a causa della scarsità di dataset disponibili e della difficoltà nell'acquisizione di dati annotati in modo coerente. Questo rappresenta un ostacolo significativo allo sviluppo di modelli robusti in questo dominio.

Nel complesso, emerge una separazione tra approcci basati su dati RGB, tecniche di analisi video e visione neuromorfica, evidenziando la mancanza di soluzioni che combinino in modo efficace questi diversi paradigmi.

1.3 Obiettivi

L'obiettivo principale di questa tesi è progettare e sviluppare una pipeline automatizzata per la generazione, l'elaborazione e l'analisi di dati sintetici, con lo scopo di addestrare reti neurali al riconoscimento delle emozioni.

In particolare, il lavoro si concentra sulla realizzazione di una pipeline in grado di:

- Generare animazioni facciali **sintetiche** a partire da **segnali audio** e convertirle in rappresentazioni di **eventi neuromorfici**, utilizzando le label dell'audio iniziale.
- Addestrare **reti neurali** utilizzando il dataset sintetico per il riconoscimento automatico delle emozioni (*emotion recognition*) utilizzando i dati **RGB** e **neuromorfici prodotti**.

2. Tecnologie utilizzate

Il codice sorgente versionato fin dall'inizio dello sviluppo si trova nella seguente repository:

https://github.com/alberto-pizzi/unreal_tesi

2.1 Software e servizi

- **JetBrains PyCharm**: IDE di sviluppo
- **Github**: per il versionamento del codice
- **Lucidchart**: per la produzione dei diagrammi UML e diagrammi pipeline

2.2 Python e librerie

- Python 3.9.25
- Numpy 2.0.2
- Ambiente *Conda* (v. 25.3.1) con relative dipendenze.

2.3 NVIDIA Audio2Face API

NVIDIA Audio2Face API è un'interfaccia di programmazione basata sull'intelligenza artificiale generativa che consente di creare animazioni facciali realistiche e sincronizzazione labiale (lip-sync) partendo semplicemente da **una sorgente audio**. Permette inoltre di dare **espressività** al volto in base al contenuto dell'audio facendo **anche** *emotion-recognition*.

In particolare, NVIDIA Audio2Face-3D è un microservizio per animare le caratteristiche facciali di personaggi 3D in modo che si adattino a qualsiasi traccia audio, sia per videogiochi, film o assistenti digitali in tempo reale. Questo modello è progettato per uso commerciale.

NVIDIA Audio2Emotion è integrato in Audio2Face ed è progettato per riconoscere automaticamente le emozioni nel parlato umano. Queste previsioni vengono utilizzate per guidare le espressioni facciali dell'avatar di Audio2Face, rendendole ancora più naturali.

Per la configurazione vedere il capitolo 5.1.2.

2.4 Unreal Engine

Unreal Engine è uno dei più avanzati e diffusi motori di gioco (game engine) 3D in tempo reale, sviluppato da Epic Games. È utilizzato per creare videogiochi di alto livello (AAA), film, simulazioni architettoniche e produzioni virtuali, grazie a potenti strumenti come Lumen (illuminazione dinamica) e Nanite (rendering geometrico).

In questo lavoro di tesi viene fatto l'uso di **MetaHuman**, ovvero esseri umani digitali foto-realistici, creati tramite un framework basato su cloud di Epic Games integrato con Unreal Engine. Consentono di generare personaggi 3D di altissima qualità, completamente "riggati" (pronti per l'animazione), inclusi capelli, vestiti e dettagli del viso, in pochi minuti.

Versione utilizzata: **Unreal Engine 5.6.1**

Plugin per Unreal:

- Python Editor Script Plugin
- Sequencer Scripting
- Movie Render Queue
- MetaHuman SDK

Per la configurazione vedere il capitolo 5.1.3.

2.5 Simulatori di eventi neuromorfici

Per far girare i seguenti le seguenti librerie/pacchetti è necessario un ambiente *Conda*.

- Frames2Ev [10]
- RPG_V2E [7]

2.6 Librerie e frameworks per Machine Learning

Il framework principale è **PyTorch**.

Poi sono stati usati in particolare:

- Ambiente *Conda* (v. 25.3.1) con relative dipendenze.
- Torch 2.8.0 [11]
- CUDA 12.8 (per GPU NVIDIA)
- Torchvision 0.23.0

2.7 Hardware

Hardware del PC locale personale utilizzato per tutto lo sviluppo, training ed alcuni test:

- OS: Windows 11 Pro (v. 25H2)
- RAM: 32 GB DDR4 2666 MHz
- CPU: AMD Ryzen 5 3600 6-core 3.59 GHz
- GPU: NVIDIA RTX 2060 SUPER (con 8 GB VRAM dedicati), compatibile con CUDA.
- SSD: Crucial P5 M2 NVMe 1TB

3. Pipeline Workflow

In questo capitolo verrà trattato il workflow complessivo della pipeline soprattutto a **livello logico**.

Nella figura 3.1 viene mostrata il workflow **complessivo** della pipeline. I nodi a forma cerchio sono i file in input/output per il modulo precedente/successivo.

Le relative implementazioni per ogni step verranno trattate nel capitolo 5 e 6.

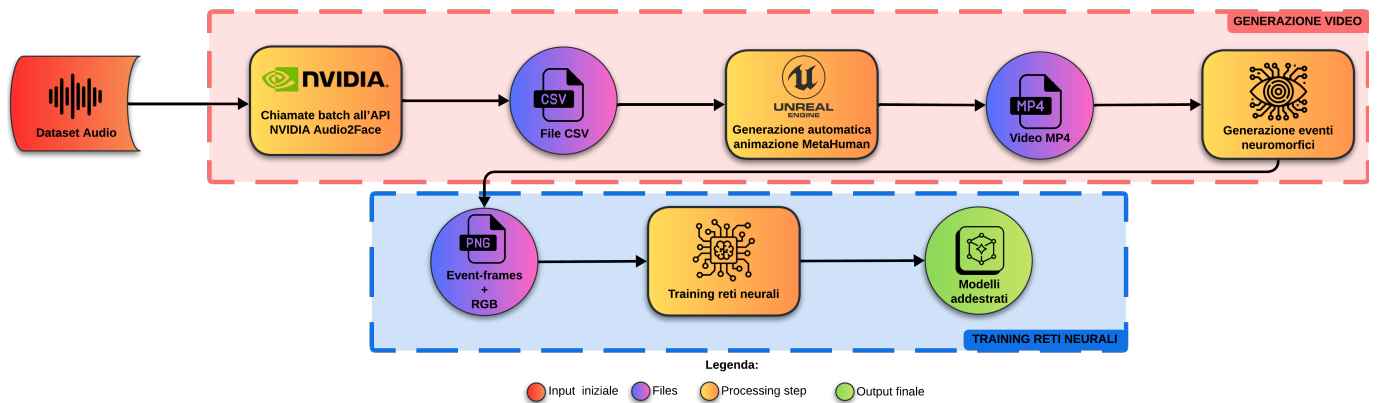


Figura 3.1: Workflow *macroscopico* della pipeline.

La pipeline è composta da **2 macro-fasi** principali: la generazione video e il training delle reti neurali. La prima *macro-fase* è divisibile in **3 blocchi** di elaborazione, dedicati alla generazione video, che si interfacciano con servizi **differenti**. La seconda *macro-fase* è composta da **1 blocco** dedicato al training delle reti neurali.

3.1 Generazione dei CSV a partire dai file audio

Nel primo blocco della pipeline i dati audio rappresentano l'input iniziale del sistema. A partire da tali segnali, viene effettuato un processo di analisi volto all'estrazione delle informazioni relative alle espressioni facciali.

Questo viene fatto tramite chiamate API ad NVIDIA Audio2Face **in batch** in modo tale che dati n audio possiamo ottenere n file CSV, quindi **un CSV per ogni audio**.

Questo modulo è molto importante visto che ha la responsabilità di interpretare i dati audio utili per poi costruire l'animazione. Quindi se dato in input viene dato un audio che rappresenta l'emozione "angry" e viene interpretato come "happy". Ne risentiranno tutti i moduli successivi a cascata perché la generazione viene fatta sulla base dell'interpretazione audio di questo modulo.



Figura 3.2: Workflow primo blocco della pipeline.

3.2 Generazione automatica video MP4 da Unreal Engine

Il secondo blocco, il più complesso e articolato della pipeline (in figura 3.3), è dedicato alla generazione automatica di sequenze video a partire dai dati contenuti nei file CSV. Questa fase viene eseguita all'interno dell'ambiente Unreal Engine e consente di trasformare i parametri numerici delle espressioni facciali in animazioni realistiche applicate a personaggi MetaHuman.

L'intero processo è completamente automatizzato e suddiviso in 4 sottofasi principali: la mappatura dei CSV, la generazione delle animazioni, l'assemblaggio delle sequenze e il rendering finale dei video. Questi aspetti saranno approfonditi nel capitolo 5.4.



Figura 3.3: Workflow secondo blocco della pipeline.

3.2.1 Generazione Animation Sequences a partire dei CSV

In questa fase, i file CSV generati precedentemente vengono utilizzati per costruire animazioni facciali sotto forma di *Animation Sequence*.

I valori dei blendshape vengono letti e mappati sui *Control Rig* facciali del MetaHuman. Per ciascun frame temporale, i coefficienti vengono convertiti in *keyframes* associati ai controlli corrispondenti, permettendo così di ricostruire l'animazione del volto nel tempo.

Successivamente, le animazioni vengono consolidate mediante un processo di *bake*, che consente di **trasformare** i keyframe del Control Rig in una *Animation Sequence* riutilizzabile e più leggera. Questo approccio permette di separare la definizione dell'animazione dalla sua applicazione ai diversi personaggi visto che i Control Rig vengono fatti sullo scheletro facciale che è **comune** a tutti i MetaHumans.

3.2.2 Creazione ed assemblaggio Level Sequence

Una volta generate le Animation Sequence, queste vengono integrate all'interno di una *Level Sequence*, che rappresenta la struttura temporale complessiva della scena.

In questa fase vengono configurati tutti gli elementi necessari alla composizione della scena, tra cui:

- il MetaHuman a cui applicare l'animazione;
- la telecamera, posizionata in modo da inquadrare correttamente il volto;
- eventuali parametri di illuminazione e ambiente.

La Level Sequence consente quindi di orchestrare l'interazione tra animazione, personaggio e scena, costituendo il contenitore principale per il rendering finale.

3.2.3 Rendering MRQ

L'ultima fase di questo blocco consiste nel rendering delle sequenze attraverso la *Movie Render Queue* (MRQ) di Unreal Engine.

Il sistema consente di esportare automaticamente le sequenze in formato video *MP4 H.264 (8 bit)*, specificando parametri quali risoluzione, frame rate e qualità dell'immagine. L'utilizzo del MRQ permette inoltre di gestire l'esecuzione in batch del rendering, rendendo possibile la generazione di un elevato numero di video in modo efficiente.

Il risultato di questa fase è un insieme di video sintetici che rappresentano le animazioni facciali generate a partire dagli input audio iniziali.

3.3 Simulazione flusso di eventi neuromorfici

Nel terzo blocco della pipeline (in figura 3.4), i video generati vengono convertiti in rappresentazioni neuromorfiche, al fine di simulare il comportamento di sensori a eventi (DVS).

Questa fase consente di **trasformare** informazioni visive dense in flussi di eventi sparsi nel tempo, che risultano particolarmente adatti per applicazioni di elaborazione efficiente e per l'addestramento di reti neurali specializzate.



Figura 3.4: Workflow terzo blocco della pipeline.

3.3.1 Divisione del video in frame

In una prima fase, ciascun video viene suddiviso in una sequenza di frame discreti (in formato PNG). Questa operazione permette di ottenere una rappresentazione esplicita dell'evoluzione temporale dell'immagine, necessaria per le successive trasformazioni.

I frame estratti costituiscono la **base** per la generazione degli eventi neuromorfici e fanno parte della **parte RGB** estratta.

3.3.2 Generazione frame neuromorfici PNG

A partire dai frame ottenuti, vengono generati i corrispondenti frame neuromorfici utilizzando simulatori di eventi (elencati nel capitolo 2.5). Tali strumenti producono immagini che rappresentano le variazioni di intensità luminosa nel tempo, codificate sotto forma di eventi.

Il risultato è una sequenza di immagini in formato PNG che descrivono l'attività neuromorfica associata al video originale. Questi dati costituiscono l'input per la fase di addestramento delle reti neurali.

3.4 Training reti neurali

Il blocco di training della pipeline (in figura 3.5) riguarda l'addestramento supervisionato (*Supervised Learning*) di modelli di apprendimento automatico per il riconoscimento delle emozioni a partire dai dati neuromorfici generati.

Questa fase utilizza i dati prodotti nei passaggi precedenti per costruire un dataset etichettato, che viene impiegato per addestrare una rete neurale in grado di classificare le espressioni facciali.

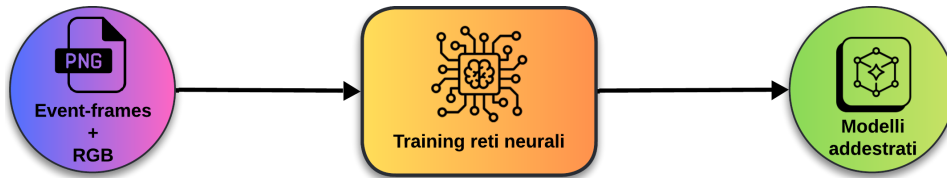


Figura 3.5: Workflow della fase di training della pipeline.

3.4.1 Creazione training directory

In questa fase, i dati neuromorfici vengono organizzati all'interno di una struttura di directory coerente con i requisiti del processo di addestramento. Maggiori dettagli verranno trattati nel capitolo 6.

Le immagini vengono suddivise in base alle **etichette associate**, che rappresentano le diverse emozioni. Tale organizzazione consente di facilitare il caricamento dei dati e la loro gestione durante il training.

3.4.2 Preparazione del dataset e avvio del training

Una volta organizzati i dati, viene avviato il processo di addestramento della rete neurale. I dati vengono caricati e suddivisi in insiemi di training e testing, permettendo di monitorare le prestazioni del modello durante l'apprendimento.

Il modello apprende a riconoscere le diverse emozioni analizzando i pattern temporali presenti RGB o neuromorfici PNG, producendo infine un classificatore in grado di generalizzare su nuovi input.

4. Datasets audio

I dataset audio utilizzati contengono dei file audio con delle brevi frasi recitate da attori, differenziandosi almeno per:

- **attore**
- **emozione**
- **frase pronunciata**
- **intensità**

L'attore, l'emozione e la frase pronunciata verranno estratti **per ogni** tipologia di dataset.

Alcuni dataset possono fornire più informazioni demografiche ad esempio: genere, etnia o età.

Queste informazioni possono essere scritte direttamente nel nome del file audio oppure in file esterni (ad esempio CSV). Se necessario, essendo il codice già predisposto, le informazioni aggiuntive possono essere estratte senza problemi.

Nel capitolo 5.2 verrà trattato più in dettaglio il modo e le logiche con cui vengono estratte queste informazioni, in base alla tipologia di dataset.

4.1 CREMA-D

Fonte bibliografica di riferimento: [3].

Lingua dataset: inglese

Formato audio: .wav

I dati principali sono scritti direttamente nel nome del file, invece altre informazioni demografiche come genere, etnia e età si trovano all'interno di un file CSV esterno.

Questo dataset contiene circa differenti attori **91 attori**, ordinati per numero (actor ID).

Ognuno dei quali ha espresso le seguenti emozioni (tra parentesi il relativo tag nel nome del file):

- Anger (ANG)
- Disgust (DIS)
- Fear (FEA)
- Happy/Joy (HAP)
- Neutral (NEU)

- Sad (SAD)

Ogni attore può pronunciare anche la frase secondo le seguenti intensità:

- Low (LO)
- Medium (MD)
- High (HI)
- Unspecified (XX)

Ogni attore può pronunciare le seguenti frasi:

- It's eleven o'clock (IEO).
- That is exactly what happened (TIE).
- I'm on my way to the meeting (IOM).
- I wonder what this is about (IWW).
- The airplane is almost full (TAI).
- Maybe tomorrow it will be cold (MTI).
- I would like a new alarm clock (IWL)
- I think I have a doctor's appointment (ITH).
- Don't forget a jacket (DFA).
- I think I've seen this before (ITS).
- The surface is slick (TSI).
- We'll stop in a couple of minutes (WSI).

Quindi un esempio del filename audio potrebbe essere: [1001_IEO_ANG_HI.wav](#)

4.2 RAVDESS

Fonte bibliografica di riferimento: [9].

Il dataset RAVDESS (Ryerson Audio-Visual Database of Emotional Speech and Song) è composto da 7356 file multimediali che rappresentano registrazioni vocali e video di attori che esprimono diverse emozioni in contesti controllati. Ogni file del dataset è identificato in modo univoco tramite una stringa numerica strutturata secondo una convenzione a sette campi, che codifica le principali caratteristiche dello stimolo emotivo e della registrazione.

Ogni filename segue il formato:

XX-XX-XX-XX-XX-XX-XX.wav

dove ciascun campo rappresenta una specifica informazione:

- **Modality:** indica la modalità del segnale.
 - 01 = full-AV (audio e video)
 - 02 = video-only
 - 03 = audio-only: in questo lavoro di tesi verrà utilizzata **solo** questa *modality*.
- **Vocal channel:** indica il tipo di produzione vocale.
 - 01 = speech: in questo lavoro di tesi verrà utilizzato **solo** questo *vocal channel*.
 - 02 = song
- **Emotion:** rappresenta l'emozione espressa nel file.
 - 01 = neutral
 - 02 = calm
 - 03 = happy
 - 04 = sad
 - 05 = angry
 - 06 = fearful
 - 07 = disgust
 - 08 = surprised
- **Emotional intensity:** indica l'intensità dell'emozione.
 - 01 = normal
 - 02 = strong

Si noti che per l'emozione *neutral* non è prevista la versione a intensità forte.

- **Statement:** identifica la frase pronunciata.
 - 01 = “Kids are talking by the door”
 - 02 = “Dogs are sitting by the door”
- **Repetition:** indica la ripetizione della frase.
 - 01 = 1st repetition
 - 02 = 2nd repetition
- **Actor:** identifica l'attore che ha effettuato la registrazione (da 01 a 24). Gli attori con identificativo dispari sono di sesso maschile, mentre quelli pari sono di sesso femminile.

Un esempio di filename è: [03-01-06-01-02-01-12.wav](#)

5. Implementazione

In questo capitolo tratteremo le implementazioni più rilevanti di alcuni script Python che riguardano tutta la pipeline **fino** al training delle reti neurali (**escluso**). Quindi tratteremo le implementazioni che riguardano la **generazione dei dati** per il training. L'implementazione e le logiche adottate per il training sono discusse nel capitolo 6.

5.1 Setup progetto e ambienti

Per eseguire gli script presenti nella repository è necessario fare il setup dei relativi ambienti.

5.1.1 Setup ambienti Python e Conda

Per il setup degli ambienti, *MiniConda* è sufficiente per questo scopo. La versione di Python installata è molto importante, perché versioni precedenti o successive a quella citata nel capitolo 2 potrebbero generare problemi.

5.1.2 Setup di NVIDIA Audio2Face API

Seguire la guida ufficiale al link della fonte [12], per scaricare il pacchetto dentro il proprio ambiente Python (anche virtuale).

5.1.3 Setup di Unreal Engine

1. Scaricare ed eseguire Unreal Engine 5.6+, preferibilmente per Windows 10/11.
2. Creare un nuovo progetto di gioco come progetto C++; un progetto vuoto è sufficiente.
3. Verificare l'installazione i plugin elencati nel capitolo 2.4 tramite [Modifica->Plugin](#). Poi riavviare l'editor, se necessario.
4. Creare o aprire un livello tramite [File->NuovoLivello](#) oppure [File->ApriLivello](#).
5. Installare o aggiungere MetaHumans al progetto tramite [Finestra->QuixelBridge](#).
6. Verificare l'aggiunta corretta dei MetaHumans controllando la presenza di una nuova cartella denominata MetaHumans in Content Browser, contenente sottocartelle con i nomi dei MetaHumans aggiunti.
7. Creare, nel percorso di gioco [/Game/Content/](#), una nuova cartella denominata Python.
8. Inserire in questa cartella tutti gli script Python necessari. Eventualmente, è possibile inserire anche i file di collegamento.
9. Aprire il file [config.py](#) e impostare i parametri:
10. Aggiungere i nomi ufficiali dei MetaHumans alla lista `METAHUMANS_INSTALLED`, prestando attenzione a maiuscole e minuscole.

11. Impostare i percorsi, **distinguendo** tra percorsi di gioco e percorsi assoluti. I percorsi di gioco iniziano sempre dalla cartella `/Game/`.
12. Verificare che ogni cartella indicata nel percorso di gioco esista effettivamente.
13. Configurare le variabili booleane iniziali per abilitare o disabilitare alcune parti della pipeline di Unreal.
14. Creare un file di risorse per le impostazioni di rendering e un file di risorse per la coda di rendering nel percorso specificato in `config.py`:
15. Aprire MRQ tramite `Finestra->Cinematiche->Codadirenderingfilmato`, impostare ed esportare le risorse nei rispettivi **percorsi di gioco**.
16. Aggiornare `config.py` con i nomi delle risorse, se non già configurati.

5.1.4 Setup di Frames2Ev

Una volta ottenuti i video .mp4 dai rendering di Unreal Engine, è necessario scomporre i video in Per prima cosa è necessari installare l'ambiente conda come mostrato nella repository [10].

Poi per eseguire Frames2Ev è possibile farlo in **uno** dei seguenti modi:

- Configurando ed eseguendo lo script `frames_to_eventframes.py`
- Seguendo le istruzioni della repository [10] eseguendo direttamente il comando.

5.2 File audio processing

In questa sezione verranno discusse l'architettura e le logiche utilizzate per il processing dei vari dataset audio in merito anche alla scalabilità e modularità del codice, in modo tale da estrarne le informazioni utili per l'elaborazione.

Nella figura 5.1 sono mostrate le dipendenze dei file che riguardano la gestione dei dataset audio, in particolare si nota che il file `audio_dataset_processing.py` è il file col quale si interfaceranno altri moduli non riguardanti l'audio, quindi per interpretare i dataset.

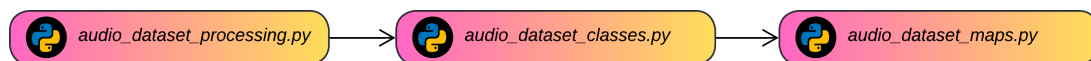


Figura 5.1: Dipendenze dei file riguardanti la gestione dei dataset audio.

Tenendo conto della struttura dei dataset audio esposti nel capitolo 4, definiamo una *dataclass* per gestire le logiche di estrazione come definito nello Snippet 5.1. Dove vengono utilizzate delle enumerazioni presenti nel file `data_enums.py`.

```

1 @dataclass
2 class AudioInfo:
3     path: Path
  
```

```

4  actor: str
5  emotion: data_enums.Emotions
6  sentence: str
7  gender: data_enums.Genders = data_enums.Genders.UNSPECIFIED
8  intensity: data_enums.Intensity = data_enums.Intensity.UNSPECIFIED
9  extra: Dict[str, Any] = None

```

Snippet 5.1: Definizione della *dataclass* `AudioInfo` per l'elaborazione dei file audio.

Progettazione Nel file `audio_dataset_classes.py`, come mostrato in figura 5.2, è presente una classe astratta `AudioDatasetParser` che è possibile estendere con delle classi figlie che rappresentano i vari dataset da processare, così da attribuire le proprie regole di interpretazione ad ognuno. Questo viene fatto con il metodo astratto `parse(...)`.

Sono stati implementate le classi di due dataset, non necessariamente quelli usati.

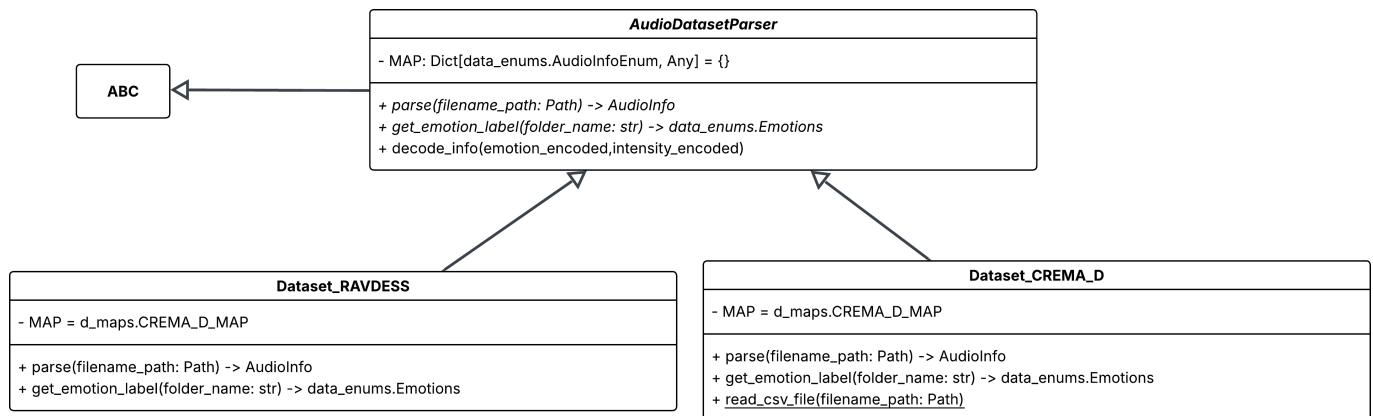


Figura 5.2: Diagramma UML dell'architettura del file `audio_dataset_classes.py`.

Nella seguente immagine viene mostrato il class diagram UML per il file `audio_dataset_classes.py`. In particolare, in base alla figura 5.1:

- `audio_dataset_classes.py`: implementa le classi parser per i dataset audio supportati. Ogni classe implementa la logica specifica di decodifica del nome file del campione audio in un'etichetta emotiva standardizzata. L'architettura è progettata per essere estesa con nuovi dataset tramite aggiunta di una nuova classe parser.
- `audio_dataset_maps.py`: contiene le enumerazioni e i dizionari di mappatura specifici per i dataset audio. Definisce le corrispondenze tra i codici numerici o alfanumerici presenti nei nomi dei file dei dataset e le etichette semantiche da estrarre per ogni dataset. Centralizza i riferimenti necessari per la decodifica dell'emozione da parte delle classi in `audio_dataset_classes.py`.
- `audio_dataset_processing.py`: contiene classi con regole e logiche per elaborare ogni tipo di dataset. Espone la funzione `get_emotion_label_for_training()` come interfaccia unificata, consentendo di combinare dataset eterogenei senza modificare il codice chiamante.

Ad esempio, l'implementazione del metodo `parse(...)` della classe `Dataset_CREMA_D` in [audio_dataset_classes.py](#) per il dataset CREMA-D, discusso nel capitolo 4, è mostrata nello snippet 5.2. Si osserva una corrispondenza con i nomi dei file tipici del dataset CREMA-D e con la logica di interpretazione implementata nel metodo. In particolare, i vari tag vengono prima separati e poi elaborati tramite la `AudioInfo` dataclass definita nello snippet 5.1. Questo approccio permette di estrarre tutte le informazioni contenute nei file ed elaborarle **interamente** in Python, facilitando la creazione di logiche personalizzate, come filtri o altre manipolazioni dei dati.

```

1  def parse(self, filename_path: Path) -> AudioInfo:
2      filename = filename_path.stem
3      split = filename.split("_")
4
5      actor_encoded = split[0]
6      sentence_encoded = split[1]
7      emotion_encoded = split[2]
8      intensity_encoded = split[3]
9
10     emotion_decoded, intensity_decoded = self.decode_info(emotion_encoded,
11                                                         intensity_encoded)
12
13     data = self.read_csv_file(Path(self.CSV_PATH))
14
15     csv_gender = data[int(actor_encoded)-1001]["Sex"]
16
17     if csv_gender in d_maps.CREMA_D_MAP[data_enums.AudioInfoEnum.GENRE]:
18         gender = d_maps.CREMA_D_MAP[data_enums.AudioInfoEnum.GENRE][
19             csv_gender]
20     else:
21         gender = data_enums.Genders.UNSPECIFIED
22
23     return AudioInfo(path=filename_path, actor=actor_encoded, emotion=
24                     emotion_decoded, sentence=sentence_encoded, intensity=intensity_decoded,
25                     gender=gender)

```

Snippet 5.2: Implementazione del metodo `parse(...)` per il dataset CREMA-D

5.3 Chiamate batch ad NVIDIA Audio2Face API

In questa sezione verrà discussa la logica utilizzata per l'ottenimento dei file CSV, contenenti i blendshape facciali per ogni frame, utili per il passo successivo della pipeline, ovvero la generazione delle animazioni facciali.

In questo caso, il file coinvolto è solamente uno, ovvero [a2f_2_csv_batch.py](#), utile per le chiamate in batch dell'*API NVIDIA Audio2Face*. In particolare, esso itera su una directory di file audio .wav, chiama l'API A2F per ciascuno e scarica il CSV risultante contenente i valori dei blendshape **AR-**

Kit per ogni frame. Può essere eseguito in qualsiasi ambiente Python con il client A2F installato, indipendentemente da Unreal Engine.

Le chiamate API vengono effettuate come descritto nella guida ufficiale [12], iterando l'operazione su un'intera directory in batch, dopo aver inserito la chiave API e scelto il modello da utilizzare tramite la *function-ID*. Poiché la cartella restituita ha un nome corrispondente alla data di esecuzione (una sorta di timestamp), essa verrà rinominata iterativamente con il nome del file audio di partenza, in modo da identificare **univocamente** il file audio di provenienza.

In particolare, ogni chiamata all'*NVIDIA Audio2Face API* genera **3 file CSV** e restituisce anche il **file audio di input** stesso all'interno di una cartella. I file sono i seguenti:

- **a2f_input_emotions.csv**: contiene eventuali override di input sulle emozioni
- **a2f_smoohthed_output.csv**: per ogni timecode viene attribuiti un valore per ogni emozione riconosciuta
- **animations_frames.csv**: questo è il file utile al nostro scopo. Esso contiene i valori per ogni blendshape ARKit (compresi tra 0 e 1) per ogni timecode dei possibili movimenti facciali rilevati dall'audio. Gli animation frames sono a **30 FPS**.
- **out.wav**: stesso file audio di ingresso che viene restituito.

Nella figura 5.3 viene mostrato un esempio della struttura del file CSV **animation_frames.csv**. La prima colonna contiene i **frame**, la seconda in **timecode** (in secondi) e le restanti colonne i **blendshape ARKit**. Questi ultimi dovranno essere mappati sui Control Rig, poiché i MetaHuman hanno rig propri per il controllo facciale, rendendo quelli ARKit incompatibili.

	A	B	C	D	***	S	T	U	V	W	X
1		timeCode	blendShapes.EyeBlinkLeft	blendShapes.EyeLookDownLeft	blendShapes.JawRight	blendShapes.JawOpen	blendShapes.MouthClose	blendShapes.MouthFunnel	blendShapes.MouthPucker	blendShapes.MouthLeft	
2	0	0.0	0.013749878853559494	0.0	0.02768438123166561	0.10403260588645935	0.0	0.0	0.02671779878437519	0.0037657672073692083	
3	1	0.03333333333333333	0.014575398527085781	0.0	0.027828237041831017	0.10614462196826935	0.0	0.0	0.031424641609191895	0.00468529485166073	
4	2	0.06666666666666667	0.015464658848941326	0.0	0.02809605561196804	0.10816264897584915	0.0	0.0	0.019694631919264793	0.0049547599628567696	
5	3	0.1	0.01664539596438408	0.0	0.02795748971402645	0.10072173178195953	0.0	0.0	0.017618875950574875	0.003845775965601206	
6	4	0.13333333333333333	0.01598917879164219	0.0	0.028295397758483887	0.10407094657421112	0.0	0.0	0.01723390631377697	0.004211889114230871	
7	5	0.16666666666666667	0.01721092127263546	0.0	0.029088472947478294	0.10322186350822449	0.0	0.0	0.008206385187804699	0.004471261985599995	
8	6	0.2	0.015476448461413383	0.0	0.028675362467765808	0.10528205335140228	0.0	0.0	0.01051065232604742	0.004038134589791298	
9	7	0.23333333333333334	0.01704035805632038	0.0	0.027399301528930664	0.0833435207605362	0.0	0.0	0.09363821893930435	0.0019472839776426554	
10	8	0.26666666666666667	0.016727721318602562	0.0	0.026973430067300797	0.05089659888801575	0.0	0.0	0.15989850461483002	0.0	
11	9	0.3	0.019119344651699066	0.0	0.02823551930487156	0.06000746786594391	0.0	0.0	0.17640239000320435	0.000971688889670372	
12	10	0.33333333333333333	0.02232963591814041	0.0	0.0290679968893528	0.031749796122312546	0.0	0.0	0.3144785165786743	0.00032416937756352127	
13	11	0.36666666666666664	0.02346116304397583	0.0	0.02830839715898037	0.04919491335749626	0.0	0.0	0.520165205001831	0.0	
14	12	0.4	0.023545779287815094	0.0	0.02698802389204502	0.11057937890291214	0.0	0.0095452219247818	0.6226204037666321	0.0	
15	13	0.43333333333333335	0.02447253093123436	0.0	0.028874525800347328	0.19082067906856537	0.0	0.06585545837879181	0.6500009894371033	0.0002467591257300228	
16	14	0.46666666666666667	0.025890490040183067	0.0	0.03342694416642189	0.3094572424888611	0.0	0.0	0.5947603583335876	0.000566472124774009	
17	15	0.5	0.02337735705077648	0.0	0.036222958064079285	0.3470008075237274	0.005162447225302458	0.009071351028978825	0.5993088483810425	5.822079401696101e-05	
18	16	0.53333333333333333	0.01925850473344326	0.0	0.03506538271903992	0.259370129108429	0.008859260939061642	0.0	0.5671947598457336	0.0013118601636961102	
19	17	0.56666666666666667	0.022993402555584908	0.0	0.02392634004354477	0.27440977096557617	0.0	0.008590874262154102	0.6264169216156006	0.00019001311738975346	
20	18	0.6	0.02361697144806385	0.0	0.017086710780858994	0.18282951414585114	0.006489238236099482	0.0	0.592574954032898	0.005115439184010029	
21	19	0.63333333333333333	0.022611821070313454	0.0	0.01679646037518978	0.10886462032794952	0.0	0.0	0.6468378901481628	0.005554277449846268	
22	20	0.66666666666666667	0.023327862843871117	0.0	0.025835800915956497	0.032065242528915405	0.0	0.0	0.5948390960693359	0.00606053089722991	
23	21	0.7	0.02156233787536621	0.0	0.031917911022901535	0.029780525714159012	0.0	0.0	0.5497179627418518	0.012582078576087952	
24	22	0.73333333333333333	0.017593661323189735	0.0	0.03458670899271965	0.07363013178110123	0.0	0.0	0.5488539338111877	0.00731104239821434	
25	23	0.76666666666666667	0.021252015605568886	0.0	0.034885309636592865	0.15988242626190186	0.00279094441793859	0.0	0.6881459951400757	0.007494783494621515	
26	24	0.8	0.023776356130838394	0.0	0.02973097376525402	0.16360019147396088	0.0	0.0	0.5834923386573792	0.0014702078187838197	
27	25	0.83333333333333334	0.02269786223769188	0.0	0.030756762251257896	0.20013582706451416	0.0	0.0	0.39004674553871155	0.0	
28	26	0.86666666666666667	0.024796241894364357	0.0	0.03727031499147415	0.2413296401500702	0.0	0.0	0.224956214427948	0.002352900847792625	
29	27	0.9	0.025270072743296623	0.0	0.03785417973995209	0.25720372796058655	0.0	0.0	0.19597181677818298	0.002237655688072114	
30	28	0.93333333333333333	0.026355639100074768	0.0	0.04084885120391846	0.2595244348049164	0.0	0.0	0.11064065992832184	0.0010602693761140108	

Figura 5.3: Esempio struttura del file **animation_frames.csv** generato da NVIDIA Audio2Face API

5.4 Generazione video animazioni con Python API in Unreal Engine

Documentazione utilizzata: [4–6].

In questa sezione verrà discussa la logica e l'architettura utilizzata per la generazione delle animazioni video in Unreal Engine, contenenti i blendshapes facciali per ogni frame, utili per il passo successivo della pipeline, ovvero la generazione delle animazioni facciali.

- `main.py`: Coordina l'intero flusso operativo di Unreal Engine: dalla raccolta dei file CSV generati da NVIDIA Audio2Face, alla creazione delle Animation Sequence per ciascun MetaHuman installato, fino alla composizione e all'importazione delle Level Sequence nel Movie Render Queue (MRQ). Gestisce inoltre la logica condizionale abilitata tramite le variabili booleane definite in `config.py`, garantendo la flessibilità nell'esecuzione parziale della pipeline.
- `config.py`: File di configurazione centrale della parte della pipeline riguardante Unreal Engine. Definisce tutte le variabili di controllo booleane (flag di attivazione/disattivazione delle fasi), i percorsi assoluti e i game path di Unreal Engine relativi a file CSV di input, asset audio, AnimSequence, Level Sequence, scheletri e MetaHuman. Contiene altresì la lista dei MetaHuman installati nel progetto (con relativo gruppo etnico, fascia d'età e genere) e i parametri della fotocamera (aspect ratio, field of view) e del rendering (preset di qualità, nome della coda MRQ).

Nella figura 5.4 è mostrato il diagramma del flusso logico dello step ingrandito riguardo la generazione automatica dell'animazione dei MetaHumans in Unreal Engine. I blocchi verdi numerati rappresentano le **fasi principali** descritte nei paragrafi successivi.

Ogni rettangolo indica un **processo logico**, che può essere svolto dalla combinazione di più funzioni di moduli diversi. I rombi indicano le decisioni dettate dai valori impostati nel `config.py` per **saltare manualmente** alcune parti della pipeline.

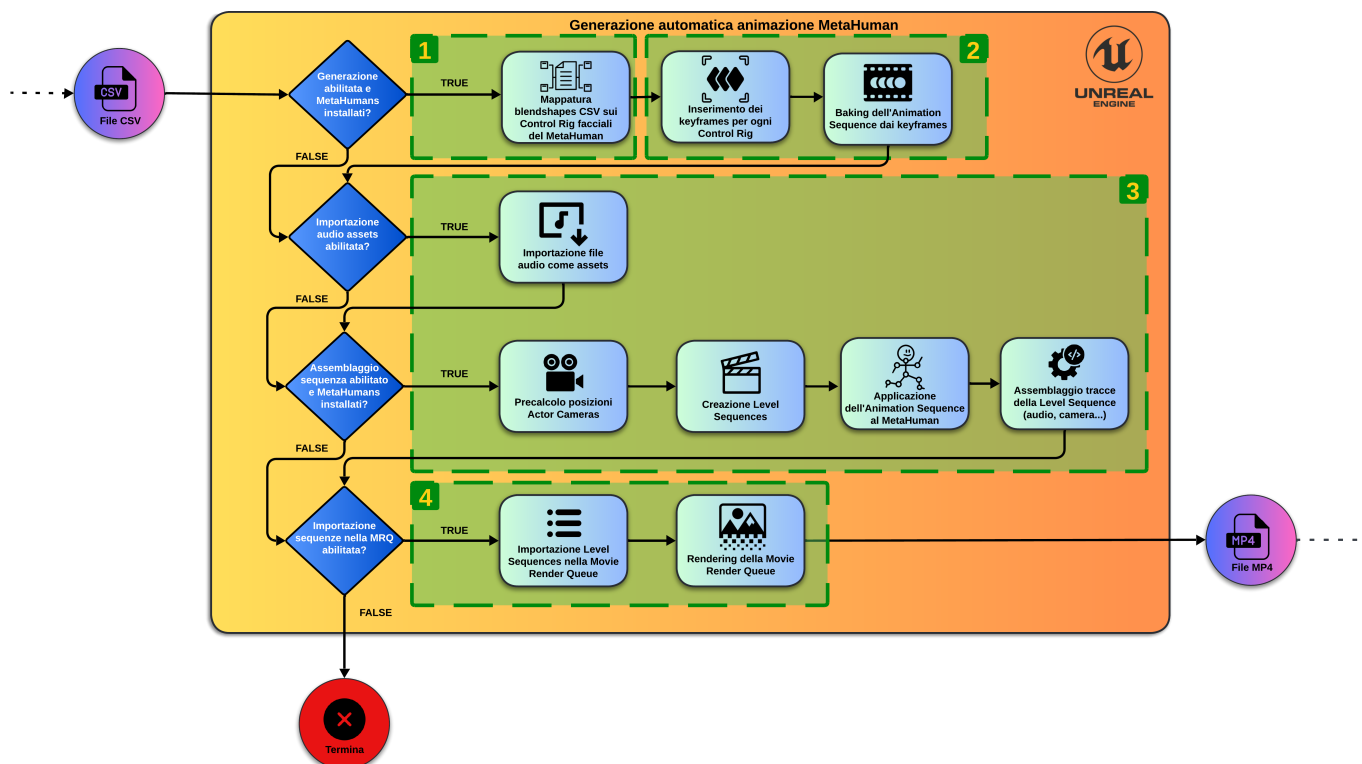


Figura 5.4: Diagramma di flusso dello step (ingrandito) di Unreal Engine.

5.4.1 Parsing CSV e mappatura blendshapes

Blocco 1 della figura 5.4

NVIDIA Audio2Face permette di generare dati di animazione facciale a partire da audio, esportando i valori dei blendshape in formato CSV. Il set standard generato include sia le 52 forme facciali ARKit definite da Apple, sia ulteriori blendshape aggiuntivi per movimenti più dettagliati di bocca, lingua e guance, per un totale di 71 colonne.

I MetaHuman in Unreal Engine, invece, utilizzano un sistema di animazione facciale basato su *control rig* e *pose asset* (insieme di valori mappati), piuttosto che su un semplice set di blendshape ARKit. Questo significa che non tutti i blendshape esportati da Audio2Face possono essere applicati direttamente a un MetaHuman.

Pertanto, per utilizzare correttamente i dati di Audio2Face con un MetaHuman è necessaria una **mappatura** tra i blendshape del CSV e i controlli del rig facciale del MetaHuman, come mostrato nello snippet 5.3. Questa mappatura consente di convertire i valori di movimento generati da Audio2Face nei controlli corrispondenti del MetaHuman, garantendo una riproduzione accurata delle espressioni facciali.

```

1 A2F_TO_METAHUMAN = {
2     # ...
3     "EyeBlinkLeft": ["CTRL_L_eye_blink"],

```

```

4     "EyeBlinkRight": ["CTRL_R_eye_blink"],
5     "CheekSquintLeft": ["CTRL_L_eye_cheekRaise", "CTRL_R_eye_faceScrunch"],
6     # ...
7 }

```

Snippet 5.3: Esempio di implementazione della mappatura dei blendshapes ARKit sui Control Rig del MetaHuman

[a2f_involved_rig_maps.py](#) contiene il dizionario *A2F_TO_METAHUMAN*, che costituisce la tavola di mappatura principale tra i blendshape ARKit prodotti da NVIDIA Audio2Face e i nomi dei control rig facciali **coinvolti** del MetaHuman in Unreal Engine. Per ogni blendshape del CSV, il dizionario elenca la lista dei rig MetaHuman che ne dipendono, gestendo così le relazioni **uno-a-molti**.

Invece, [control_rig_maps.py](#) contiene il dizionario di **override** dei control rig, che associa il nome del controllo MetaHuman alla classe Python corrispondente definita in [control_rig_classes.py](#). Questo file rappresenta il registro dei controlli non-standard, ovvero tutti quei rig che richiedono una logica di calcolo personalizzata anziché una mappatura diretta uno-a-uno.

Nel diagramma UML in figura 5.5 è rappresentata la classe **ControlRig**, che corrisponde a un rig di **default** di tipo **RigType.FLOAT**, ossia con valore monodimensionale. Tuttavia, la mappatura è necessaria perché la corrispondenza tra rig non è sempre uno-a-uno, ma può essere uno-a-molti: alcuni rig dei MetaHuman possono avere valori multidimensionali, ad esempio di tipo *RigType.VEC2*, ossia array bidimensionali (x, y).

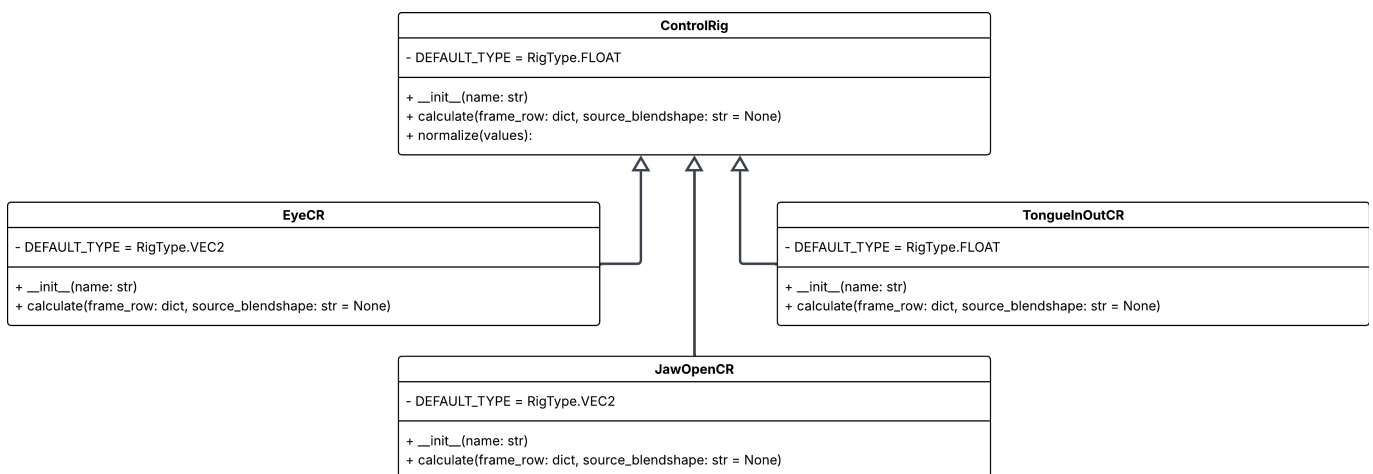


Figura 5.5: Diagramma UML dell'architettura del file [control_rig_classes.py](#).

Per gestire questi casi, è possibile estendere la classe “di default” **ControlRig** e **alcune** classi figlie, definendo il nuovo tipo di rig (**RigType**) e il metodo di calcolo dei valori tramite `calculate(...)`, che assegna i dati presenti nel CSV al rig corrispondente in base al suo tipo. Uno snippet di esempio è riportato nello snippet 5.4.

```

1 class EyeCR(ControlRig):
2     DEFAULT_TYPE = RigType.VEC2
3
4     def __init__(self, name):

```

```

5         super().__init__(name)
6
7     def calculate(self, frame_row: dict, source_blendshape: str = None):
8         x = frame_row.get("EyeLookOutLeft", 0) - frame_row.get("EyeLookInLeft",
9         0)
10        y = frame_row.get("EyeLookUpLeft", 0) - frame_row.get("EyeLookDownLeft",
11        0)
12        return x, y

```

Snippet 5.4: Implementazione della classe EyeCR in `control_rig_classes.py`

Alcuni rig, pur avendo lo stesso tipo, possono essere soggetti a una relazione molti-a-uno: più blendshape del CSV possono corrispondere a un singolo rig. Questo approccio è illustrato, ad esempio, dalla classe `TongueInOutCR` nello snippet 5.5.

```

1 class TongueInOutCR(ControlRig):
2     DEFAULT_TYPE = RigType.FLOAT
3
4     def __init__(self, name):
5         super().__init__(name)
6
7     def calculate(self, frame_row: dict, source_blendshape: str = None):
8         x = frame_row.get("TongueIn", 0) - frame_row.get("TongueOut", 0)
9         return x

```

Snippet 5.5: Implementazione della classe `TongueInOutCR` in `control_rig_classes.py`

L'elaborazione delle mappe dei Control Rig viene effettuata in `control_rig_processing.py`, creando le opportune corrispondenze per poter elaborare i dati. In particolare, verifica che per i blendshape che corrispondono in modo uno-a-molti con i Control Rigs, vengano associati anche i relativi rig di override, garantendo la molteplicità dei valori.

5.4.2 Generazione Animation Sequences

Blocco 2 della figura 5.4

In questa fase vengono generate tutte le *Animation Sequences*, ovvero le animazioni applicate sulla *Face Skeletal Mesh*, effettuando le seguenti operazioni **solamente su un MetaHuman**. Questo perché tutti i MetaHumans hanno la stessa *Face Skeletal Mesh* comune. Quindi basterà generare **un'animazione per ogni CSV** ottenuto.

Per fare ciò verrà creata una singola Level Sequence **temporanea**, dove avviene lo *spawn* temporaneo del MetaHuman così da generarci l'Animation Sequence. La Level Sequence viene pulita ogni volta che è stata generata una **singola** animazione.

`csv_to_face_animation.py`: Gestisce l'intera catena di trasformazione dal dato CSV alla Animation Sequence ottenuta tramite bake sui keyframes sui Control Rig, in Unreal Engine.

Inserimento keyframes

La fase di inserimento dei keyframes consiste nella traduzione dei dati contenuti nei file CSV in animazioni facciali all'interno di Unreal Engine. In particolare, ogni riga del CSV rappresenta un istante temporale (frame), mentre le colonne contengono i valori dei diversi blendshape.

In una prima fase, viene individuato il Control Rig facciale associato alla Level Sequence, selezionando il rig denominato *Face_ControlBoard_CtrlRig*. Questa operazione è necessaria per poter applicare correttamente i valori ai controlli facciali.

Successivamente, vengono estratti i frame temporali a partire dalla prima colonna del CSV, che rappresenta i frames. Tali frame definiscono la timeline dell'animazione.

Per ottimizzare il processo, i dati vengono riorganizzati in una struttura batch, raggruppando i valori per ciascun Control Rig. Questo approccio consente di ridurre il numero di chiamate alle API di Unreal e di migliorare l'efficienza durante l'inserimento dei *keyframes*.

Infine, i valori vengono applicati alla Level Sequence attraverso una funzione dedicata che assegna, per ogni frame e per ogni controllo, il corrispondente valore numerico. Al termine del processo, viene impostato automaticamente l'intervallo di playback della sequenza in base all'ultimo frame disponibile, garantendo la corretta e precisa riproduzione dell'animazione.

Baking dell'Animation Sequence

Una volta inseriti i keyframes nella Level Sequence, viene eseguita la fase di *baking*, il cui obiettivo è convertire l'animazione procedurale basata su Control Rig in un asset statico di tipo *AnimSequence*.

Il processo inizia con l'individuazione del *binding* associato al volto del MetaHuman all'interno della sequenza. Successivamente, viene caricato lo scheletro di riferimento, necessario per la creazione dell'asset di animazione.

Tramite una factory (*AnimSequenceFactory*), viene inizializzato un nuovo oggetto *AnimSequence*, specificando il percorso di salvataggio e il nome dell'asset. Vengono inoltre configurate le opzioni di esportazione, abilitando sia le trasformazioni sia i morph target, in modo da preservare tutte le informazioni relative alle espressioni facciali.

La fase di *baking* vera e propria viene eseguita tramite la funzione `export_anim_sequence(...)`, che campiona i valori dei controlli nel tempo e li converte in *keyframe* standard dell'animazione. Questo passaggio rende l'animazione indipendente dal Control Rig e quindi riutilizzabile su altri personaggi compatibili, in modo più rapido e leggero.

Infine, l'asset generato viene salvato nel Content Browser di Unreal Engine. Questo consente di **riutilizzare** le Animation Sequence all'interno della pipeline, ad esempio per il rendering di nuovi video o per l'addestramento delle reti neurali.

5.4.3 Gestione Sequencer

Blocco 3 della figura 5.4

La gestione del **Sequencer** in Unreal Engine rappresenta un passaggio cruciale per la creazione di scene cinematografiche e animazioni complesse. Il Sequencer consente di coordinare elementi come attori, telecamere, animazioni e tracce audio in una timeline temporale, permettendo il controllo preciso del loro comportamento e della loro sincronizzazione.

In questa fase tutte le Animation Sequences prodotte vengono **combinare iterativamente** con tutti i MetaHumans installati, ovvero presenti nella lista `METAHUMANS_INSTALLED`.

Il modulo `sequencer_manager.py` è stato sviluppato per semplificare l'interazione con il Sequencer, offrendo funzioni per automatizzare la creazione e la gestione delle **Level Sequence**. Grazie a queste funzioni, è possibile aggiungere e configurare attori, telecamere, animazioni facciali e corporee, oltre a tracce audio, riducendo notevolmente l'intervento manuale e migliorando l'efficienza nella produzione di contenuti cinematografici all'interno di Unreal Engine.

La gestione del sequencer comprende dunque operazioni di creazione, configurazione e pulizia delle sequenze, oltre alla gestione di asset e parametri associati agli attori e alle telecamere, garantendo una pipeline coerente e ripetibile per la produzione di scene complesse.

I seguenti paragrafi sono organizzati per passi logici, seguendo il diagramma in figura 5.4.

Importazione audio come assets

Il modulo supporta l'importazione di file audio come asset Unreal e la loro integrazione nella sequenza tramite `import_audio_as_asset`. L'audio viene importato nella relativa cartella contenente gli audio importati definita nel `config.py`. Questi assets saranno utili per l'assemblaggio del tracce.

Precalcolo posizioni Actor Cameras per le inquadrature

Nel contesto della gestione del sequencer, è fondamentale conoscere in anticipo le posizioni e le rotazioni delle telecamere rispetto agli attori, per garantire inquadrature frontali corrette e coerenti con la scena. Questa precauzione è necessaria perché le posizioni degli scheletri possono essere ottenute solo da elementi di tipo *possessable*, ossia attori già presenti e controllabili nel livello. Tuttavia, tutte le operazioni implementate nel modulo utilizzano elementi di tipo *spawnable*, generati automaticamente dal Sequencer al momento della riproduzione della sequenza, come se fossero temporanei. In questo modo, non è necessario gestire manualmente la loro distruzione, semplificando la gestione della scena e delle risorse.

Il modulo `sequencer_manager.py` implementa questa funzionalità tramite la funzione `get_actor_cams_locations_and_rotations`, che calcola le coordinate dell'Actor Camera rispetto all'attore. In particolare viene presa la posizione della testa, ed in base ad un offset viene posizionata di conseguenza. Questo è necessario perché non tutti i MetaHuman hanno le stesse conformazioni anatomiche

(altezza, ecc...).

In base all'attore, il sistema:

- Genera (in modo *possessable*) temporaneamente l'attore nella scena.
- Calcola la posizione e la rotazione della telecamera, tipicamente considerando la testa o un punto di riferimento rilevante del modello.
- Memorizza le coordinate calcolate in una struttura dati per essere riutilizzate successivamente durante la costruzione della sequenza.
- Rimuove l'attore generato, evitando modifiche permanenti alla scena.

Questo precalcolo è essenziale per pianificare le inquadrature prima della generazione effettiva della sequenza, garantendo che ogni telecamera sia correttamente posizionata e orientata rispetto agli attori.

Creazione e gestione Level Sequences

Il modulo fornisce strumenti per creare nuove **Level Sequence** tramite la funzione `create_level_sequence` oppure per caricare sequenze esistenti. I percorsi degli asset sono gestiti attraverso costanti definite in configurazione, il che permette di organizzare facilmente le sequenze e i relativi asset senza interventi manuali. Questa funzionalità costituisce la base su cui costruire e manipolare la timeline della scena, dato che verrà creata **una Level Sequence per ogni combinazione** (MetaHuman + Animation Sequence).

Applicazione Animation Sequence al MetaHuman

L'applicazione di sequenze di animazione (generata dal *bake*) ai MetaHuman è un passaggio fondamentale per animare le espressioni facciali e i movimenti del corpo all'interno di una Level Sequence. Nel modulo `sequencer_manager.py`, questa operazione viene gestita principalmente tramite la funzione `attach_anim_sequence_to_face(...)`, che permette di collegare una sequenza di animazione facciale a un MetaHuman presente nella sequenza.

Il procedimento prevede i seguenti passaggi:

- Individuazione del binding facciale del MetaHuman nella sequenza, attraverso il nome della traccia (standard) corrispondente.
- Caricamento della sequenza di animazione desiderata come asset Unreal.
- Impostazione del range di playback della sequenza in base alla durata dell'animazione e al frame rate della Level Sequence.
- Creazione di una traccia di animazione facciale (*Skeletal Animation Track*) e associazione della sequenza caricata a questa traccia.

- Aggiornamento della Level Sequence e salvataggio dell'asset, rendendo la sequenza pronta per la riproduzione.

Questo approccio garantisce che le animazioni facciali siano sincronizzate con la timeline complessiva della sequenza e che il MetaHuman possa esprimere movimenti naturali e coerenti, senza richiedere interventi manuali per la gestione dei keyframe. La stessa logica può essere estesa, con adattamenti, anche alle animazioni del corpo.

Assemblaggio tracce della Level Sequence

L'assemblaggio delle tracce all'interno di una Level Sequence rappresenta uno dei passaggi principali per costruire una scena cinematografica in Unreal Engine. In questa fase, il modulo `sequencer_manager.py` coordina l'inserimento e la configurazione di tutte le componenti della sequenza, includendo attori, telecamere, animazioni e tracce audio.

In questa fase **tutti** gli elementi attore (attori e Actor Cameras) sono di tipo *spawnable*. Le operazioni principali comprendono:

- **Aggiunta di attori:** gli attori vengono inseriti nella sequenza e associati alle rispettive tracce, in modo da controllarne posizione, rotazione e visibilità lungo la timeline.
- **Inserimento delle telecamere:** le telecamere possono essere aggiunte e posizionate rispetto agli attori, con parametri come *field of view* e *aspect ratio* configurati dal `config.py`. I tagli di camera (*camera cuts*) vengono impostati per controllare quali inquadrature siano attive in ciascun intervallo di tempo.
- **Inserimento traccia audio:** il relativo file audio importato come asset viene inserito nella sequenza, con l'intervallo di riproduzione sincronizzato automaticamente con il range di playback (impostato durante l'applicazione dell'Animation Sequence al MetaHuman, capitolo 5.4.3)
- **Rimozione tracce dei Control Rig facciali:** una volta applicata l'Animation Sequence vengono rimosse le tracce dei Control Rig per alleggerire il Sequencer.
- **Salvataggio:** una volta completato, salva la Level Sequence nella directory impostata in `config.py` e procede con altre eventuali iterazioni delle combinazioni.

In sintesi, questa fase permette di assemblare in maniera coerente e automatizzata tutti gli elementi della *Level Sequence*, garantendo che attori, animazioni, telecamere e audio siano sincronizzati lungo la timeline e pronti per la riproduzione della scena.

5.4.4 Rendering

Blocco 4 della figura 5.4

Il modulo `rendering.py` fornisce strumenti per gestire l'importazione delle Level Sequences nella **Movie Render Queue** (MRQ) di Unreal Engine e per configurare le operazioni di rendering automatizzato. L'obiettivo principale è facilitare la preparazione e l'esecuzione dei rendering senza intervento manuale nell'editor.

Le funzionalità principali del modulo comprendono:

- **Gestione dei percorsi degli asset:** vengono ottenuti i percorsi completi dei diversi asset utilizzati per la sequenza e le impostazioni di rendering, semplificando l'accesso via codice.
- **Gestione della coda MRQ:** funzioni come `clear_all_mrqr()` e `get_current_queue(...)` consentono di visualizzare e ripulire la coda di rendering, garantendo che le sequenze vengano elaborate senza conflitti o *job* residui.
- **Importazione delle sequenze nella MRQ:** la funzione `import_ls_into_mrqr(...)` permette di aggiungere una Level Sequence alla coda di rendering, associando il relativo livello di scena e applicando automaticamente le impostazioni di rendering specificate in un preset Unreal preventivamente creato.
- **Configurazione automatica dei job:** ogni sequenza importata viene configurata con un preset di rendering predefinito, che controlla parametri come risoluzione, codec e formato di output, garantendo uniformità e ripetibilità tra diversi rendering.

In sintesi, il modulo consente di preparare e gestire in maniera automatica le sequenze (Level Sequences) per il rendering in Unreal Engine, automatizzando operazioni che altrimenti richiederebbero interventi manuali nell'interfaccia del Movie Render Queue. Quindi chiamando semplicemente `import_ls_into_mrqr(...)` verrà importata **una singola** Level Sequence nella MRQ, di conseguenza basterà iterare.

Successivamente potrà essere avviato il rendering da Unreal Engine con un solo click. In questo modo otterremo dei video RGB secondo la configurazione desiderata, ad esempio formato *MP4 H.264 (8 bit)*.

5.5 Generazione eventi neuromorfici

Una volta generati i video RGB con Unreal Engine, il passo successivo è la conversione di questi dati in una rappresentazione neuromorfica. Questo processo simula il funzionamento delle **camere event-based**, o sensori **DVS** (*Dynamic Vision Sensor*), che rilevano variazioni di intensità luminosa nel tempo, anziché acquisire frame completi a intervalli regolari. Questi sensori sono particolarmente utili in condizioni di scarsa illuminazione.

La conversione viene effettuata in due fasi principali: la scomposizione del video in frame e la successiva generazione degli eventi neuromorfici a partire da tali frame.

5.5.1 Scomposizione dei video in frames

I video generati in formato MP4 vengono inizialmente suddivisi in una sequenza ordinata di frame RGB. Questa operazione è necessaria in quanto i simulatori neuromorfici operano su sequenze di immagini statiche piuttosto che su flussi video compressi.

La scomposizione viene effettuata utilizzando script Python `video_to_frames.py` che utilizza il comando in batch `ffmpeg`, che permettono di estrarre ogni frame mantenendo l'ordine temporale originale di tutta la directory. Sarebbe necessario garantire rappresentazione temporale sufficientemente densa per la simulazione degli eventi.

I frame vengono salvati in formato PNG per evitare perdite di qualità dovute alla compressione. Vengono organizzati in directory strutturate per ogni video, facilitando le successive fasi di elaborazione batch e mantenendo una chiara corrispondenza tra video originale e sequenza di immagini.

5.5.2 Generazione eventi neuromorfici dai frames

Una volta ottenute le sequenze di frame, si procede con la simulazione del flusso di eventi neuromorfici per ciascuna sequenza. Questo processo prevede la generazione di eventi asincroni in corrispondenza delle variazioni di intensità luminosa tra frame consecutivi.

Per ogni pixel, si calcola la variazione di luminosità nel tempo. Quando tale variazione supera una soglia predefinita, viene generato un evento con coordinate spaziali, timestamp e polarità (incremento o decremento di intensità).

La generazione degli eventi avviene tramite strumenti di simulazione neuromorfica, che consentono di trasformare sequenze RGB in rappresentazioni *event-based*. Il risultato è una sequenza di frame neuromorfici (o stream di eventi) che mette in risalto principalmente le variazioni dinamiche dell'immagine, riducendo le informazioni ridondanti presenti nei frame statici.

Questa rappresentazione è particolarmente adatta al riconoscimento delle emozioni, in quanto enfatizza i movimenti facciali nel tempo, che rappresentano l'informazione chiave per distinguere le diverse espressioni.

Inoltre, durante la generazione degli eventi neuromorfici, il sistema applica eventualmente un meccanismo di *upsampling* adattivo. Questo significa che, se il framerate del video originale non è sufficientemente elevato per catturare correttamente le variazioni di intensità luminosa tra frame consecutivi, il sistema inserisce dinamicamente frame intermedi. Tale operazione permette di aumentare la risoluzione temporale della sequenza, garantendo che gli eventi generati rappresentino fedelmente i movimenti rapidi del volto o di altre parti in movimento.

L'*upsampling* adattivo è fondamentale per preservare la coerenza temporale della simulazione neuromorfica: senza di esso, movimenti veloci potrebbero non essere rilevati correttamente, causando perdite di informazioni o artefatti nel flusso di eventi. Inoltre, questa tecnica consente di mantenere un equilibrio tra precisione della simulazione e quantità di dati generati, evitando di saturare il sistema con un numero eccessivo di eventi quando non necessario.

In particolare, *Frames2Ev* [10] genera **2 tipi** di file per ogni video scomposto in frame:

- **event-frames PNG** trattati come RGB
- **file NPZ**: formato usato da *NumPy* per salvare più array in un unico file compresso. È essenzialmente un contenitore (simile ad un file “zip”) di array *NPY*. Non utilizzato in questa pipeline.

Quindi verranno utilizzati solamente gli event-frames PNG prodotti, per l’addestramento in modo che si adattino facilmente al codice di training (capitolo 6).

6. Training reti neurali

Documentazione ufficiale (versione citata nel capitolo 2): [11]

6.1 Reti neurali utilizzate

Per il riconoscimento delle emozioni da video RGB e flussi neuromorfici, sono stati utilizzati diversi modelli di reti neurali, scelti per la loro capacità di catturare informazioni spaziali e temporali complesse.

- **ResNet-18 (base)**: rete convoluzionale 2D che estrae feature significative da singoli frame. Per il nostro obiettivo, permette di rappresentare le caratteristiche facciali rilevanti in ogni frame, come espressioni e movimenti dei muscoli facciali.
- **ResNet-18 + LSTM**: combina la ResNet-18 con un'unità LSTM. La ResNet-18 estrae le feature spaziali dai frame, mentre l'LSTM cattura le dipendenze temporali tra i frame. Questo è fondamentale per riconoscere emozioni che si manifestano come sequenze di movimenti facciali nel tempo.
- **R3D_18 (Conv3D)**: utilizza convoluzioni tridimensionali che analizzano simultaneamente le dimensioni spaziali e temporali del video. Per il nostro scopo, permette di rilevare pattern dinamici delle espressioni facciali senza dover trattare i frame singolarmente.
- **MViT_v2_s (Video Transformer)**: modello basato su attenzione che valuta le relazioni tra tutti i frame di un video. Questo approccio è particolarmente utile per catturare movimenti facciali complessi e per distinguere emozioni simili, poiché considera l'intera sequenza nel contesto temporale.

Tutti i modelli scelti sono **non lineari**, capaci di rappresentare relazioni complesse tra input e classi di emozioni, e permettono di sfruttare al meglio i dataset sintetici generati dalla pipeline per il training.

6.2 Setup

1. Installare l'ambiente conda importando `training_environment.yml`
2. Aprire `training.py`
3. Impostare i percorsi all'inizio del file
4. Impostare le trasformazioni in `get_strasform()` o creare nuove trasformazioni, se necessario

5. Accedere alla funzione `main` (nello stesso file)
6. Attivare o chiamare `create_training_directory` all'inizio di `main`, per creare directory etichettate per training e testing
7. Impostare *sampling_type*
8. Impostare la chiamata a `DataLoader` con i propri parametri: *dataset*, *batch_size* e *shuffle*
9. Impostare il modello, scegliendo la classe modello corretta
10. Impostare il tipo di loss (*criterion*) e l'optimizer
11. Impostare *num_epochs*
12. Chiamare la funzione corretta in base al tipo di operazione: `train_model(...)`, `evaluate_model(...)` o `make_inference(...)`
13. Disattivare o rimuovere le linee di codice inutili

6.3 Iperparametri

Per addestrare reti neurali temporali su video RGB e flussi di eventi neuromorfici, è fondamentale selezionare una funzione di perdita e un ottimizzatore appropriati. In questo lavoro verranno utilizzati *CrossEntropyLoss* come funzione di perdita e *Adam* come ottimizzatore; i dettagli saranno illustrati di seguito.

6.3.1 CrossEntropyLoss

CrossEntropyLoss è adatta a problemi di classificazione multi-classe, come il riconoscimento delle emozioni. Essa misura quanto la distribuzione predetta dalla rete si discosta dalla classe corretta. La formula per un singolo campione è:

$$\mathcal{L}_{CE} = - \sum_{c=1}^C y_c \log(\hat{y}_c) \quad (6.1)$$

dove:

- C è il numero di classi,
- y_c è 1 se la classe corretta è c , 0 altrimenti,
- $\hat{y}_c$ è la probabilità predetta per la classe c dalla rete.

Motivi per cui è indicata nel nostro caso:

- **Gestione diretta delle classi multiple:** la loss confronta le probabilità predette con la classe corretta, utile quando ogni video o flusso neuromorfico rappresenta una tra diverse emozioni.

- **Penalizzazione proporzionale agli errori:** assegna gradienti più grandi quando la predizione è lontana dalla classe corretta, aiutando il modello a correggere confusione tra emozioni simili. Infatti la funzione

$$-\log(x)$$

cresce molto rapidamente quando

$$x \rightarrow 0$$

Se il modello predice una probabilità bassa per la classe corretta, ad esempio

$$\hat{y}_{\text{vera}} \approx 0.01$$

la loss diventa molto grande, spingendo il modello a correggere l'errore.

Se si usasse solo la probabilità lineare, non ci sarebbe questa **penalizzazione esponenziale degli errori gravi**.

- **Compatibilità con probabilità predette (softmax):** integra softmax e log-loss, garantendo gradiente stabile e aggiornamento efficace dei pesi.
- **Funziona con reti non lineari e dati sequenziali:** supporta architetture complesse come LSTM, Conv3D e Video Transformer, permettendo di apprendere relazioni spazio-temporali senza compromettere la stabilità del training.

6.3.2 Adam

Adam è un ottimizzatore efficiente che adatta il learning rate per ciascun parametro, combinando le proprietà di *Momentum* e *RMSProp*. I suoi vantaggi principali sono:

- convergenza stabile anche con gradienti rumorosi tipici dei video;
- ridotta sensibilità alla scelta iniziale del learning rate;
- adatto a dataset sintetici e sequenze temporali lunghe.

6.3.3 Batch size

Il batch size utilizzato è pari a 8.

Tale scelta è stata guidata principalmente dalla dimensione ridotta del dataset (48 video) e dalla necessità di bilanciare stabilità del processo di ottimizzazione e capacità di generalizzazione del modello. Un batch size contenuto consente di aggiornare i pesi del modello più frequentemente durante l'addestramento, introducendo una maggiore variabilità nei gradienti e contribuendo a ridurre il rischio di overfitting. Inoltre, tale valore risulta compatibile con i vincoli computazionali legati all'addestramento di modelli video, che richiedono un elevato utilizzo di memoria GPU (hardware utilizzato, al capitolo 2.7).

6.3.4 Learning rate

Il learning rate è stato impostato a $1 \cdot 10^{-4}$. Tale valore è stato scelto per garantire un processo di ottimizzazione stabile durante il training. Un learning rate troppo elevato potrebbe infatti causare aggiornamenti eccessivi dei pesi del modello, rendendo il processo di apprendimento instabile e portando a una possibile mancata convergenza. Al contrario, un valore più contenuto consente un apprendimento graduale e più controllato, particolarmente importante nel contesto di un dataset ridotto, in cui il rischio di overfitting è elevato.

6.3.5 Consecutive Sampling dei frame

Come modalità di campionamento dei frame è stato usato approccio *consecutivo*, quindi *consecutive sampling*, per la selezione dei frame all'interno di ciascun video. Tale strategia consiste nel campionare una sequenza di frame contigui, preservando quindi l'ordine temporale originale delle informazioni visive.

La selezione della sottosequenza avviene a partire da un indice iniziale scelto in modo casuale all'interno del video, in modo da introdurre variabilità tra le diverse epoche e ridurre il rischio di apprendimento di pattern dipendenti da posizioni fisse nella sequenza.

Nel caso in cui la lunghezza del video risulti inferiore al numero massimo di frame richiesti dal modello, viene applicata una strategia di *padding*, al fine di ottenere comunque una sequenza di lunghezza costante. Questo consente di garantire l'uniformità delle dimensioni degli input, requisito necessario per l'addestramento dei modelli utilizzati.

Sebbene non rappresenti un iperparametro in senso stretto, questa fase di campionamento risulta fondamentale poiché definisce la struttura temporale dell'input e influisce direttamente sulle informazioni disponibili per l'apprendimento delle reti neurali.

6.3.6 Considerazioni finali

Nel complesso, la scelta degli iperparametri adottati è risultata coerente con le caratteristiche del problema affrontato e con le limitazioni del dataset disponibile. La combinazione tra ottimizzatore Adam, funzione di loss CrossEntropyLoss, learning rate ridotto e batch size contenuto ha permesso di ottenere un processo di addestramento stabile e controllato.

In particolare, l'insieme di queste scelte ha favorito un compromesso efficace tra capacità di apprendimento del modello e riduzione del rischio di overfitting, aspetto particolarmente rilevante considerando la dimensione limitata del dataset e la complessità delle architetture utilizzate.

Nel complesso, gli iperparametri selezionati hanno garantito una convergenza abbastanza regolare del training e hanno permesso ai modelli di apprendere rappresentazioni significative dai dati, mantenendo un comportamento coerente tra i diversi esperimenti condotti.

6.4 Creazione struttura directory

Le directory di training e testing vengono create con il metodo `create_training_directory(...)`, che organizza l'intero dataset specificato (indicando il tipo di dataset per il processing). Il dataset viene suddiviso in due cartelle:

- **train:** dove sono presenti i frame-video per il training
- **test:** dove sono presenti i frame-video per il test, indicando il dataset di testing contenente dati **diversi** dal training.

In ognuna delle quali crea una cartella **per ogni emozione** (*label*) rilevata.

All'interno di ogni cartella *label*, vengono inseriti i video come *symlink*, in modo da non copiare il video che potrebbe essere pesante su un dataset più grandi.

A seguito delle organizzazioni delle training directory e testing directory, verranno analizzate per dare i parametri di input ai modelli, come il numero di classi basato sulle label.

6.5 Progettazione e implementazione

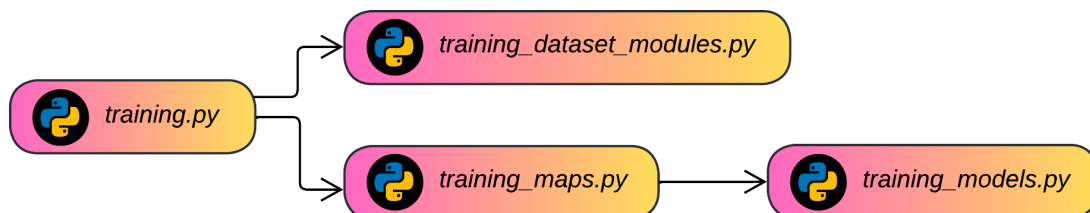


Figura 6.1: Diagramma dipendenze di tutti i file di training.

Nella figura 6.1 è possibile vedere il diagramma di tutte le dipendenze.

training.py: script principale per l'addestramento, la valutazione e l'inferenza dei modelli di riconoscimento delle emozioni. Le sue responsabilità includono: la creazione della struttura di directory del training set con symlink per classe emotiva (`create_training_directory()`); la costruzione del DataLoader con FrameDataset; l'istanziamento e il trasferimento su GPU del modello selezionato; il ciclo di addestramento con salvataggio automatico del miglior checkpoint (`train_model()`); la valutazione dell'accuratezza sul test set (`evaluate_model()`); e l'inferenza su nuove sequenze video con output dell'emozione predetta e del livello di confidenza (`make_inference()`).

Invece, **training_maps.py**, contiene il dizionario *model_map* che associa ogni valore dell'enum *ModelType* alla corrispondente classe di modello definita in **training_models.py**. Funge da factory registry, permettendo a **training.py** di istanziare dinamicamente il modello corretto con la funzione

`build_model(...)` del file `training.py`, come mostrato nello snippet 6.1.

```

1 model_map = {
2     ModelType.RESNET: ResNetFrameModel,
3     ModelType.RESNET_LSTM: ResNetLSTMModel,
4     ModelType.CONV3D: Conv3DModel,
5     ModelType.VIDEO_TRANSFORMER: VideoTransformerModel
6 }

```

Snippet 6.1: Dizionario `model_map` per la mappatura dei modelli alla loro relativa classe.

`training_dataset_modules.py`: Implementa il dataset PyTorch `FrameDataset`, responsabile del caricamento e della preparazione dei frame video per l'addestramento della rete neurale. Gestisce la scoperta ricorsiva delle cartelle per classe (corrispondenti alle etichette emotive), il campionamento dei frame secondo 3 strategie configurabili, l'applicazione delle trasformazioni torchvision e la restituzione di tensori nel formato $[T, C, H, W]$ atteso dai modelli. Fornisce anche la funzione ausiliare `load_video_frames()` per l'inferenza su singola sequenza.

`training_models.py`: Implementa i modelli di rete neurale profonda disponibili per il riconoscimento delle emozioni a partire da sequenze di frame. L'architettura è mostrata in figura 6.2. Quindi è possibile aggiungere altre reti neurali semplicemente aggiungendo una nuova classe estendendo `torch.nn.Module`.

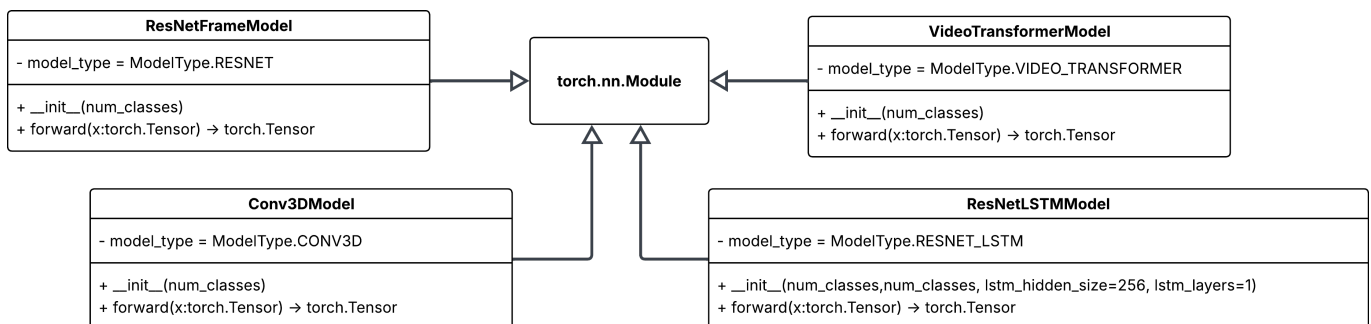


Figura 6.2: Diagramma UML dell'architettura dei modelli utilizzati.

Come mostrato nel diagramma in figura 6.2, ogni classe di modello definisce la struttura della rete, applicando le trasformazioni e i layer all'interno del metodo `__init__()`. Il metodo `forward(...)` si occupa invece della gestione dei tensori, preparando e rendendo compatibili i dati per l'elaborazione da parte del modello. Grazie a questa organizzazione, è possibile aggiungere o modificare facilmente le reti neurali, mantenendo una gestione dei tensori chiara e coerente. Questa flessibilità è una caratteristica fondamentale della classe genitore `torch.nn.Module`.

6.6 Gestione dei tensori per le reti neurali

Il dataset utilizzato produce i video come sequenze di frame rappresentate da tensori di dimensione $[T, C, H, W]$, dove T indica il numero di frame per video, C il numero di canali, H l'altezza e W la

larghezza del frame. Per poter utilizzare questi tensori come input nei diversi tipi di reti neurali, è necessario adattare la loro forma a seconda dell'architettura.

Le eventuali elaborazioni dei tensori vengono fatte nei metodi `forward(...)` della relativa classe del modello, come mostrato nel diagramma in figura 6.2.

Reti Convolutional 2D (CNN) Le CNN standard, come **ResNet-18**, operano su singoli frame. Pertanto, i tensori devono essere “appiattiti” lungo la dimensione temporale T , combinando batch e frame. La forma risultante per l'input della rete diventa:

$$[B \times T, C, H, W]$$

dove B è la dimensione del batch. Dopo la propagazione dei frame attraverso la CNN, le feature ottenute possono essere ristrutturare in $[B, T, F]$, con F numero di feature estratte per frame, e aggregate lungo T tramite media o altre strategie di pooling.

Reti Convolutional 3D (Conv3D) Le reti Conv3D, come **R3D-18**, trattano direttamente la dimensione temporale come parte della convoluzione. I tensori devono quindi avere la forma:

$$[B, C, T, H, W]$$

dove la dimensione temporale T diventa parte dell'input della rete. Questo permette al modello di catturare simultaneamente le informazioni spaziali e temporali senza ulteriori reshaping.

Reti CNN + LSTM Per le architetture che combinano CNN e LSTM, come **ResNet-18 + LSTM**, ogni frame viene prima trasformato tramite la CNN per estrarre feature di dimensione F , ottenendo un tensore $[B, T, F]$. Questo tensore diventa l'input dell'LSTM, che modella le dinamiche temporali tra i frame. La struttura dei dati per l'LSTM è quindi:

$$[Batch, Sequenza, Feature]$$

e permette alla rete di apprendere le relazioni temporali tra le feature dei frame.

Video Transformer I Video Transformer, come **MViT v2-s**, ricevono in input sequenze di frame in forma $[B, C, T, H, W]$, dove B è il batch, C i canali, T il numero di frame e H, W le dimensioni spaziali. Se il dataset produce i frame in $[T, C, H, W]$, è sufficiente una permutazione per adattarli al modello.

Il modello trasforma automaticamente ogni frame in un vettore di feature, generando un tensore $[B, T, F]$, su cui il Transformer applica l'attenzione per catturare le relazioni temporali. La testa di classificazione finale restituisce le predizioni sulle classi di interesse, senza bisogno di ulteriori reshaping manuali.

Riepilogo Il seguente schema riassume i reshaping principali, effettuati su ogni rete usata, partendo da un tensore $[T, C, H, W]$ prodotto dal dataset:

Rete Neurale	Forma input finale
ResNet-18	$[B \times T, C, H, W]$
ResNet-18 + LSTM	$[B, T, F]$
R3D-18 (Conv3D)	$[B, C, T, H, W]$
MViT v2-s (Video Transformer)	$[B, C, T, H, W]$

Questa organizzazione consente di mantenere coerenza tra i dati del dataset e le diverse architetture, preservando le informazioni spaziali e temporali necessarie per l'apprendimento delle emozioni dai video.

6.7 Dataset utilizzato

Viene utilizzato un **unico dataset** di partenza, impiegato per la realizzazione di **due distinti esperimenti**: il primo basato su input **RGB** e il secondo basato su **event frames** simulati. In entrambi i casi, la struttura del dataset e la suddivisione dei dati sono rimaste invariate, al fine di garantire un confronto coerente tra le diverse architetture e modalità di rappresentazione.

Il dataset utilizzato per il training è composto da **48 video** (generati della fase precedente della pipeline) partendo dagli audio del dataset di CREMA-D (capitolo 4), suddivisi in **2 classi** (*angry* e *happy*) corrispondenti a differenti emozioni. Per ogni video, i frame vengono estratti tramite un approccio di *consecutive sampling* (capitolo 6.3.5) con finestra temporale fissata a **16 frame**.

L'addestramento dei modelli è stato eseguito per un totale di **30 epoche**, utilizzando un batch size pari a **8**.

Per la **fase di test** sono stati invece utilizzati **16 video complessivi**. Di questi, **8 non** sono mai stati osservati durante la fase di training e presentano sia animazioni che personaggi *MetaHuman* completamente differenti rispetto ai dati di addestramento. I restanti 8 video, pur condividendo gli stessi personaggi *MetaHuman* presenti nel training set, contengono animazioni differenti, al fine di valutare la capacità del modello di generalizzare a variazioni dinamiche pur mantenendo identità visive simili.

Questa configurazione consente di valutare le prestazioni del modello sia in scenari completamente nuovi sia in condizioni parzialmente viste durante l'addestramento.

6.8 Inferenza

La fase di inferenza consiste nell'utilizzo dei modelli addestrati per la classificazione di **nuovi** video (sequenze di frames) **non** osservati durante il training. In particolare, per ciascun modello viene

caricato il checkpoint corrispondente alla migliore epoca, selezionata sulla base delle prestazioni ottenute durante l'addestramento.

Per fare ciò viene chiamato il metodo `make_inference(...)` iterativamente che riceve in input il modello addestrato e la directory del video su cui fare inferenza.

Ogni video di input viene preprocessato seguendo la stessa pipeline utilizzata in fase di training, includendo il ridimensionamento dei frame e il campionamento tramite *consecutive sampling* con finestra temporale fissa. I frame così ottenuti vengono convertiti in tensori e forniti in input al modello.

Il modello produce in output un vettore di punteggi per ciascuna classe, dal quale viene ricavata la predizione finale selezionando la classe con **probabilità massima** (*argmax*). La confidenza della predizione può essere ottenuta applicando una funzione di *softmax* agli output del modello.

Per la valutazione delle prestazioni, l'inferenza viene eseguita sull'intero set di test, consentendo il calcolo di metriche come la **confidence**, che misura l'affidabilità della classificazione ottenuta, e il **tempo medio di inferenza**, quest'ultimo ottenuto come **media aritmetica** dei tempi di elaborazione dei singoli video.

Alcuni risultati sono mostrati nella tabella in figura 7.16 e in figura 7.17.

7. Risultati

I risultati esposti in questo paragrafo sono prodotti utilizzando CREMA-D come dataset (capitolo 4). Tutti i frames generati e illustrati hanno una risoluzione di **1280x720** pixel a **30 fps**.

7.1 Risultati generazione video RGB sintetici da Unreal

Nelle seguenti foto sono mostrati alcuni dei risultati ottenuti. L'illuminazione dipende dal livello di Unreal utilizzato che è facilmente cambiabile, poi è necessario generare nuovamente i video.



Figura 7.1: Frame RGB di un'animazione del MetaHuman "Zeva" con emozione **"angry"**.



Figura 7.2: Frame RGB di un'animazione del MetaHuman "Zeva" con emozione **"happy"**.



Figura 7.3: Frame RGB di un'animazione del MetaHuman "Myles" con emozione **"angry"**.



Figura 7.4: Frame RGB di un'animazione del MetaHuman "Myles" con emozione **"happy"**.

7.2 Risultati generazione event-frames dai video

Nelle seguenti foto sono rappresentate le simulazioni neuromorfiche in PNG ottenute utilizzando *Frames2Ev* [10]. Si nota infatti che vengono captati solamente le variazioni di luminosità nel tempo per

ogni pixel rispetto al corrispettivo frame RGB. Quindi ogni pixel reagisce quando la luminosità cambia abbastanza, registrando dati solamente quando succede qualcosa (cambiamento o movimento).

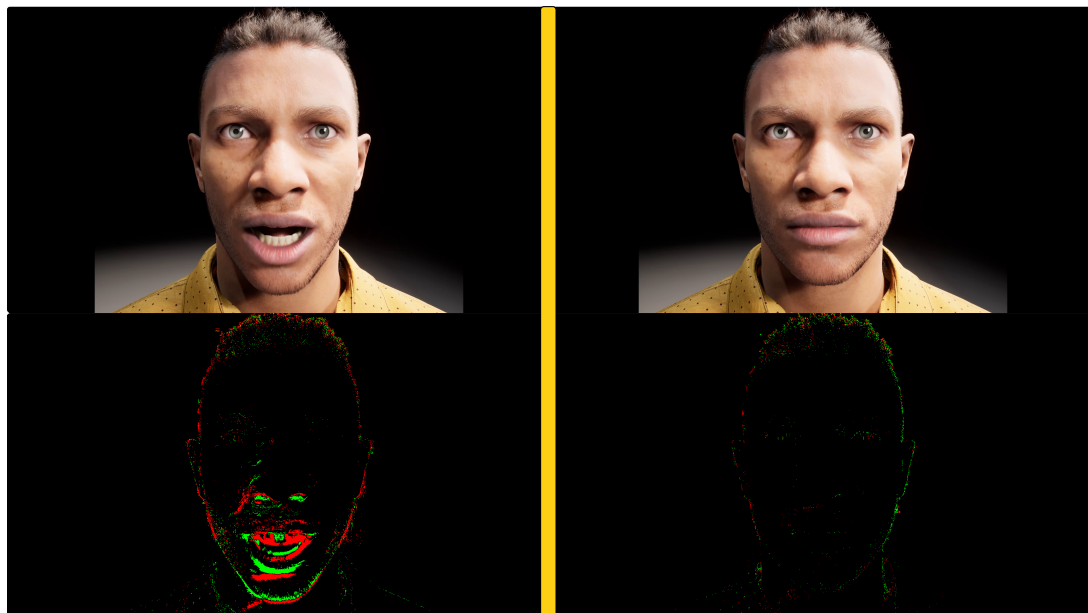


Figura 7.5: Event-frames a confronto con frames RGB di un'animazione del MetaHuman "Bryan" con emozione "**angry**".

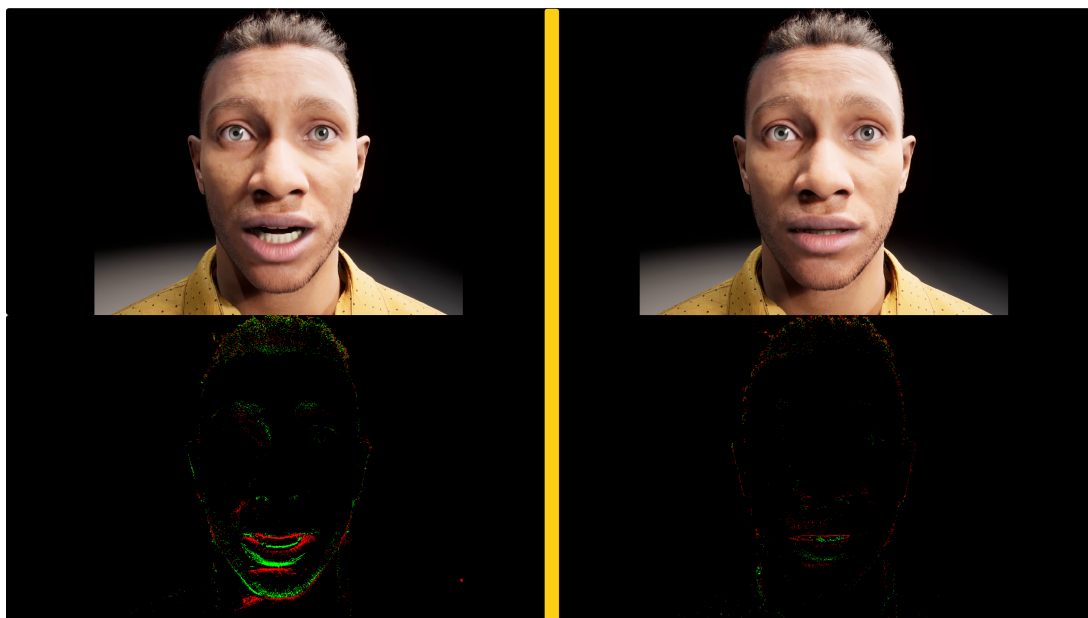


Figura 7.6: Event-frames a confronto con frames RGB di un'animazione del MetaHuman "Bryan" con emozione "**scared**".

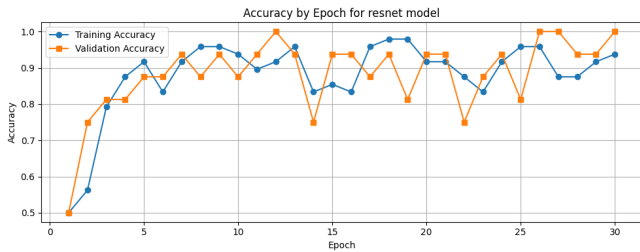
Gen. 56 video	Generazione animazioni	Assemblaggio sequenza	Rendering	Simulazione eventi	Totale
Tempo impiegato	≈ 5 minuti	≈ 3 minuti	≈ 10 minuti	≈ 17 minuti	≈ 35 minuti

Figura 7.7: Tabella dei tempi generazione di un piccolo dataset da 56 video (RGB + Event-frames)

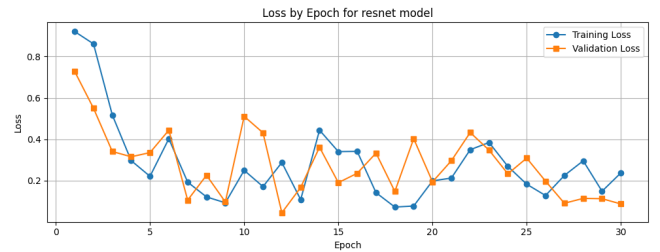
7.3 Risultati training

7.3.1 Grafici

Dati RGB

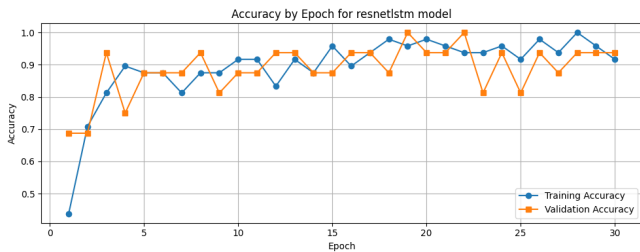


(a) Accuracy per epoca

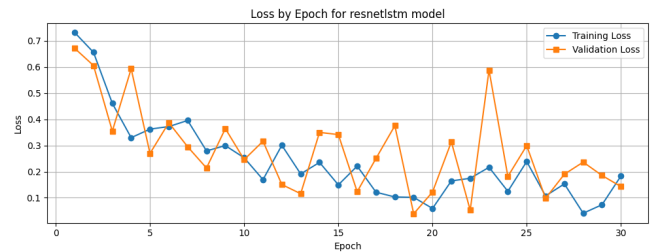


(b) Loss per epoca

Figura 7.8: Grafico con statistiche di training e testing: **ResNet18** con dati **RGB**

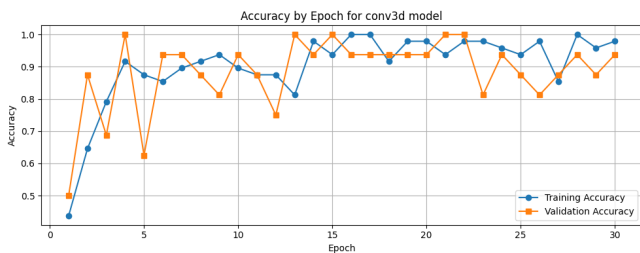


(a) Accuracy per epoca

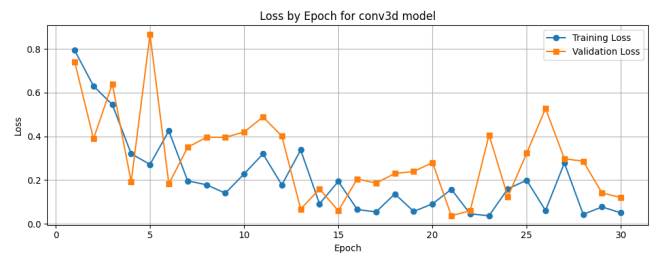


(b) Loss per epoca

Figura 7.9: Grafico con statistiche di training e testing: **ResNet18+LSTM** con dati **RGB**

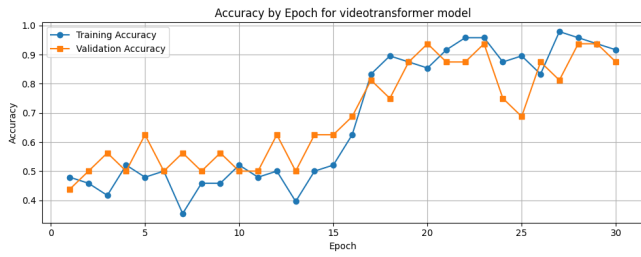


(a) Accuracy per epoca

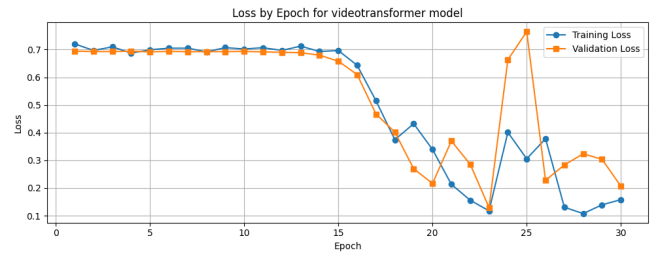


(b) Loss per epoca

Figura 7.10: Grafico con statistiche di training e testing: **R3D-18 (Conv3D)** con dati **RGB**



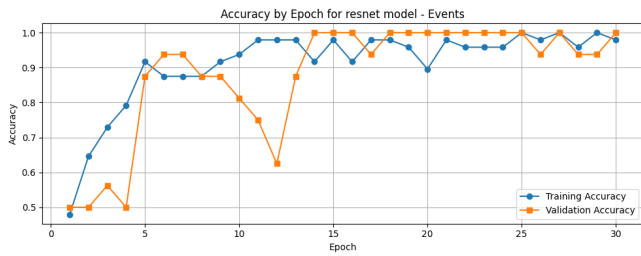
(a) Accuracy per epoca



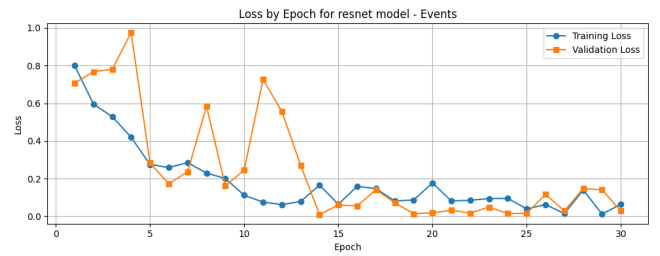
(b) Loss per epoca

Figura 7.11: Grafico con statistiche di training e testing: **MViT v2-s (Video Transformer)** con dati **RGB**

Dati neuromorfici

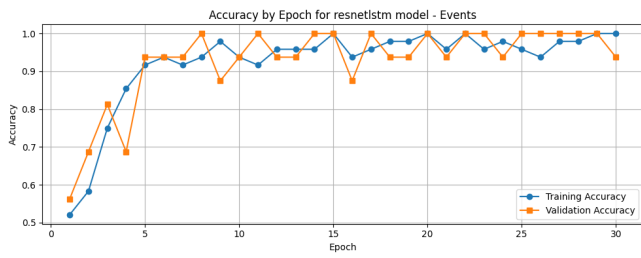


(a) Accuracy per epoca

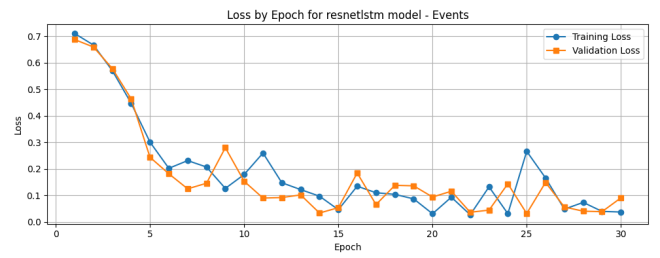


(b) Loss per epoca

Figura 7.12: Grafico con statistiche di training e testing: **ResNet18** con dati **neuromorfici**

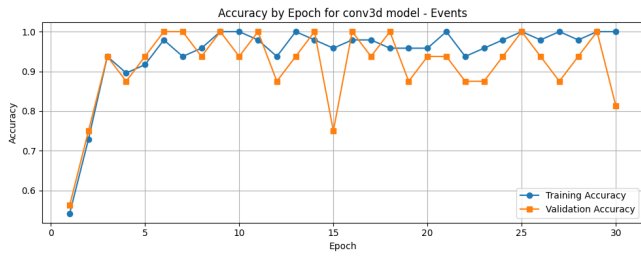


(a) Accuracy per epoca

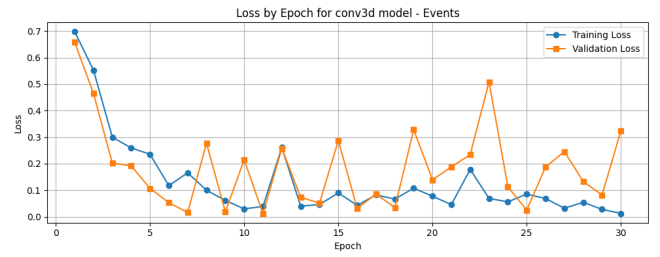


(b) Loss per epoca

Figura 7.13: Grafico con statistiche di training e testing: **ResNet18+LSTM** con dati **neuromorfici**

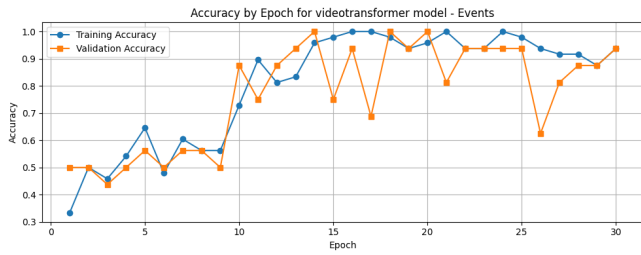


(a) Accuracy per epoca

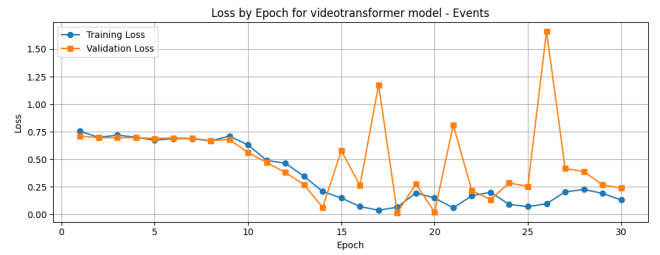


(b) Loss per epoca

Figura 7.14: Grafico con statistiche di training e testing: **R3D-18 (Conv3D)** con dati **neuromorfici**



(a) Accuracy per epoca



(b) Loss per epoca

Figura 7.15: Grafico con statistiche di training e testing: **MViT v2-s (Video Transformer)** con dati **neuromorfici**

7.3.2 Tempi

Le seguenti tabelle rappresentano **2 diversi training**, uno con i frames RGB e l'altro con gli event-frames. In particolare mostrano il tempo di training per ogni rete neurale e di inferenza media effettuata su **1 video**.

L'inferenza media è stata calcolata facendo una media aritmetica su ogni inferenza effettuata su ogni video del dataset di test, ovvero nel dataset che non contiene video utilizzati durante il training. Le inferenze sono effettuate con il modello migliore ottenuto durante il training ad ogni epoca. (informazioni sul dataset al capitolo 6.7).

Tempo impiegato	ResNet-18 (base)	ResNet-18 + LSTM	3D ResNet-18	MViT v2-s
Training	5,9598 minuti	6,1371 minuti	11,2391 minuti	119,2693 minuti
Inferenza media (1 video)	447,5075 ms (93,14 %)	470,4847 ms (90,38 %)	779,2483 ms (94,03 %)	1012,1512 ms (95,18 %)

Figura 7.16: Tabella dei tempi di training e inferenza media con dati **RGB** per ogni rete neurale.

Tempo impiegato	ResNet-18 (base)	ResNet-18 + LSTM	3D ResNet-18	MViT v2-s
Training	5,1923 minuti	5,1185 minuti	10,0338 minuti	128,0801 minuti
Inferenza media (1 video)	488,2239 ms (97,03 %)	484,4732 ms (93,71 %)	845,8194 ms (97,76 %)	1113,4865 ms (96,26 %)

Figura 7.17: Tabella dei tempi di training e inferenza media con dati **event-based** per ogni rete neurale.

7.4 Analisi e discussione

7.4.1 Analisi risultati generazione video RGB

Nella tabella in figura 7.7, vengono mostrati i tempi impiegati per la generazione di un piccolo dataset di 56 video dai quali poi verranno estratti 8 video da aggiungere al validation set per il testing (capitolo 6.7). Vengono mostrati i tempi per ciascuno step compiuto dal codice secondo i blocchi verdi mostrati nel diagramma in figura 5.4.

Nel complesso la generazione dei video è abbastanza rapida, ma onerosa in termini di memoria e GPU perché Unreal Engine è molto pesante. Quindi aumentando il numero di video da generare in batch, è decisamente necessario aumentare anche l'hardware di conseguenza.

7.4.2 Analisi risultati generazione event-frames

Per quanto riguarda gli event-frames generati, lo sfondo nero dei frames originali non genera eventi. Se fosse stato utilizzato uno sfondo illuminato, anche minime variazioni di luce (ad esempio riflessi, shading, animazioni) verrebbero registrate come eventi. In altre parole, gli eventi dipendono solo dalle variazioni e non dal colore o luminosità costante.

Facendo riferimento ancora alla tabella in figura 7.7, la simulazione richiede un po' di tempo perché verrà effettuato l'upsampling.

7.4.3 Analisi risultati training

Innanzitutto è necessario contestualizzare i risultati ottenuti al dataset utilizzato, ovvero ridotto (48 video), bilanciato, e con animazioni abbastanza omogenee per ogni label o classe.

Durante la fase di addestramento dei modelli, si è osservato che la loss media diminuisce progressivamente nel corso delle epoche, indicando che i modelli stanno effettivamente apprendendo a distinguere le emozioni presenti nei dati. L'accuratezza, valutata su un validation/testing set separato (capitolo 6.7), ha raggiunto valori abbastanza elevati, nonostante il numero ridotto di video a disposizione. Questo risultato è in parte spiegato dalla natura delle animazioni utilizzate: pur essendo generati da soggetti differenti, le espressioni principali delle emozioni *angry* e *happy* sono abbastanza marcate e

coerenti.

Un aspetto interessante riguarda l'importanza relativa delle diverse regioni del volto. Nel dataset considerato, le emozioni sono principalmente riflesse nella parte superiore del volto, come occhi e sopracciglia, mentre le variazioni labiali sono meno rilevanti. Questo può ridurre l'informazione utile disponibile per modelli che analizzano i frame singolarmente, come la ResNet senza LSTM, potenzialmente “disorientando” il training. Al contrario, modelli che catturano la dinamica temporale, come la ResNet con LSTM, Conv3D e VideoTransformer, possono sfruttare anche i micro-movimenti del labiale, migliorando la capacità di generalizzazione.

Infine, si segnala che i frame utilizzati sono codificati come immagini RGB PNG, anche quando si tratta di event-frames simulati (pur rappresentando eventi). Lo sfondo nero dei frame originali non genera eventi, mentre variazioni di illuminazione ambientale avrebbero prodotto ulteriori eventi, modificando leggermente il segnale fornito ai modelli. Queste considerazioni devono essere tenute presenti nell'interpretazione dei risultati e nella valutazione delle performance dei diversi modelli.

Analisi dei tempi di training e inferenza

I risultati ottenuti sui due dataset, basati rispettivamente su frame RGB e event frames (figure 7.16 e 7.17), evidenziano differenze sia in termini di *confidence* sia di comportamento computazionale delle diverse architetture.

Dal punto di vista delle prestazioni, gli esperimenti mostrano un miglioramento consistente della *confidence* quando vengono utilizzati gli event frames. In particolare, la ResNet-18 passa da una *consistency* media del 93.14% su RGB al 97.03% su event frames. Analogamente, la ResNet-18 con LSTM migliora da 90.38% a 93.71%, mentre la 3D ResNet-18 raggiunge il valore più alto pari al 97.76% con event frames rispetto al 94.03% su RGB. Anche il modello MViT v2-s mostra un incremento, seppur più contenuto, passando dal 95.18% al 96.26%.

Questo comportamento suggerisce che la rappresentazione basata su eventi fornisce informazioni più discriminative rispetto ai frame RGB tradizionali, probabilmente grazie alla riduzione di informazioni ridondanti e alla maggiore enfasi sulle variazioni dinamiche dell'espressione facciale.

Per quanto riguarda i tempi di training, si osserva che gli event frames tendono a ridurre leggermente il tempo di addestramento nei modelli meno complessi, come ResNet-18 e ResNet-18 con LSTM, mentre nei modelli più complessi come MViT v2-s si registra un incremento del tempo di training (128.08 minuti rispetto a 119.27 minuti). Questo suggerisce che architetture basate su Transformer possono essere più sensibili alla natura della rappresentazione in input.

Anche i tempi di inferenza mostrano un comportamento coerente tra i due dataset, con i modelli RGB generalmente più veloci rispetto agli event frames. Le differenze, tuttavia, risultano contenute e non compromettono significativamente l'efficienza complessiva dei modelli. In particolare, la ResNet-18 rimane il modello più efficiente in termini di inferenza, mentre MViT v2-s risulta il più costoso computazionalmente in entrambe le configurazioni, soprattutto contestualizzando al piccolo

dataset utilizzato il risultato è coerente.

Complessivamente, i risultati indicano che gli event frames possono essere una rappresentazione più efficace per il task di riconoscimento delle emozioni, migliorando le performance di tutti i modelli testati. Tuttavia, l'incremento di confidence non sempre si traduce in una riduzione dei costi computazionali, evidenziando un trade-off tra accuratezza ed efficienza a seconda dell'architettura utilizzata.

Andamento delle metriche di training e testing

Dati RGB

ResNet-18 Come mostrato nei grafici in figura 7.8, il modello converge rapidamente: già attorno all'epoca 5 la validation accuracy supera il 90%, e si stabilizza tra 88–100% per tutto il resto del training. La loss di training scende in modo regolare fino a valori attorno a 0.07–0.15 nelle ultime epoche. La validation loss però rimane molto rumorosa (oscillazioni tra 0.05 e 0.50 anche dopo l'epoca 20), il che indica che il modello generalizza bene in media ma è sensibile alla varianza del piccolo validation set (16 video $\times$ numero di classi). Nessun chiaro segnale di overfitting: le due curve di accuracy rimangono vicine per tutto il training.

ResNet-18+LSTM Come mostrato nei grafici in figura 7.9, il comportamento è simile a ResNet18 ma con una convergenza leggermente più lenta nelle primissime epoche (epoca 1: training accuracy $\approx$ 44% contro il 56% del ResNet puro). Dopo l'epoca 5 le curve si allineano e il modello raggiunge performance del tutto paragonabili: validation accuracy stabile tra 88–100%, con picchi a 1.0 attorno alle epoche 22 e 28. La loss di validazione mostra un picco anomalo a $\approx$ 0.59 all'epoca 23, probabilmente causato da un batch di validazione particolarmente difficile. Nel complesso il modulo LSTM non porta un vantaggio misurabile su questo dataset: le emozioni sono costanti per tutto il video, quindi la dipendenza temporale che LSTM dovrebbe catturare è già ben rappresentata dall'analisi spaziale del ResNet su un singolo frame.

R3D-18 Come mostrato nei grafici in figura 7.10, tra i tre modelli CNN il r3d-18 converge più rapidamente e raggiunge la validation accuracy più alta nelle prime epoche, con un picco a 1.0 già all'epoca 4. Dopo l'epoca 15 la validazione si attesta stabilmente tra 94–100%. La loss di training scende in modo consistente fino a $\approx$ 0.04 (la più bassa di tutti i modelli), ma la validation loss oscilla considerevolmente: si notano picchi a 0.86 (epoca 5) e 0.81 (epoca 23) che rappresentano episodi di instabilità. Questo è il comportamento tipico di un modello pre-addestrato su video (Kinetics) che si adatta molto bene a un dataset piccolo e omogeneo, ma rimane sensibile alla composizione delle clip di validazione.

MViT v2-s Come mostrato nei grafici in figura 7.11, questo è il modello che si differenzia nettamente dagli altri, con un andamento chiaramente diviso in due fasi.

- *Fase 1 (epoche 1–14)*. La loss rimane praticamente ferma attorno a 0.70, e l'accuracy oscilla tra 40–63% senza migliorare. Questo è il segnale tipico di un modello che fatica ad adattarsi al nuovo task, probabilmente perché i pesi pre-addestrati (su Kinetics-400) non sono compatibili con la struttura del problema senza un periodo di riscaldamento adeguato, oppure perché il tasso di apprendimento iniziale è troppo alto e altera eccessivamente i pesi pre-addestrati.
- *Fase 2 (epoche 15–30)*. Il modello inizia finalmente a migliorare con decisione: la loss di training scende da 0.65 a 0.11 in poche epoche, e l'accuracy raggiunge il 90%+ in training e ≈ 87 –94% in validation. Il lungo periodo iniziale senza miglioramenti pesa molto su un dataset piccolo. La validation accuracy finale ($\approx 87\%$) è la più bassa del gruppo, anche se il divario con gli altri modelli si è ridotto molto nelle ultime epoche.

Considerazioni generali Tutti e tre i modelli CNN (ResNet18, ResNet18+LSTM, r3d-18) raggiungono performance molto buone su questo dataset, il che è coerente con la natura del task: video sintetici di MetaHuman con emozioni statiche, dataset bilanciato, e campionamento di 16 frame consecutivi da un video con espressione costante. In questo scenario il contesto temporale aggiuntivo (LSTM, convoluzioni 3D) non dà un vantaggio misurabile rispetto al semplice ResNet applicato ai singoli frame. MViT v2-S dimostra invece che i transformer video richiedono molti più dati o strategie di adattamento più sofisticate (blocchi iniziali congelati, periodo di riscaldamento lungo, tassi di apprendimento differenziati per le varie parti del modello) per esprimere il loro potenziale su dataset piccoli. L'alta irregolarità della validation loss in tutti i modelli è una conseguenza diretta del validation set di soli 16 video: anche un singolo campione classificato male sposta la metrica di 6.25 punti percentuali.

Dati neuromorfici

ResNet-18 Come mostrato nei grafici in figura 7.12, la convergenza è rapida come nella versione RGB, ma con un comportamento anomalo della validation nelle epoche 4–13: si vedono picchi di loss molto alti (0.98 all'epoca 4, 0.73 all'epoca 11, 0.57 all'epoca 12) accompagnati da bruschi cali dell'accuracy fino a 0.50 e 0.63. Questo suggerisce che il modello attraversi una fase di adattamento instabile ai dati neuromorfici, che hanno una distribuzione molto diversa da quella RGB su cui ResNet18 è stato pre-addestrato. Dopo l'epoca 14 la situazione si stabilizza completamente: la validation accuracy si attesta a 1.0 o 0.94 per quasi tutte le epoche rimanenti, e la loss scende sotto 0.05. Il risultato finale è ottimo, e leggermente migliore rispetto alla versione RGB in termini di stabilità nelle ultime epoche.

ResNet-18+LSTM Come mostrato nei grafici in figura 7.13, questo è il modello più regolare di tutti su dati event-based. Le curve di training e validation sono quasi sovrapposte per tutto l'arco delle 30 epoche: discesa regolare della loss da 0.70 iniziale fino a ≈ 0.03 –0.05 finale, e accuracy che sale rapidamente a 90%+ già dall'epoca 5 e si mantiene tra 94–100% fino alla fine. Non si osservano picchi anomali né segni di sovra-adattamento. Il modulo LSTM sembra trarre vantaggio in modo più diretto dalla struttura temporale dei dati neuromorfici rispetto a quanto faceva con i dati RGB: gli

eventi hanno una rappresentazione temporale intrinsecamente sparsa e asincrona, e la cella ricorrente riesce a integrarne meglio l'informazione nel tempo.

R3D-18 Come mostrato nei grafici in figura 7.14, la convergenza è molto rapida: già all'epoca 3 la validation accuracy supera il 94%, e le due curve rimangono allineate quasi perfettamente per le prime 12 epoche. Dopo l'epoca 15 si notano oscillazioni della validation loss più marcate rispetto agli altri modelli (picchi a 0.29, 0.33, 0.51 nelle epoche 15–24), con corrispondenti cali dell'accuracy fino a 0.75 e 0.88. La training loss continua a scendere fino a ≈ 0.01 , il che segnala una leggera tendenza al sovra-adattamento nelle fasi finali: il modello memorizza il training set ma fatica a mantenere la stessa stabilità sul validation. Nel complesso le performance restano buone (94–100% di accuracy finale), ma il divario tra training e validation nella loss è il più ampio tra i quattro modelli addestrati con dati event-based.

MViT v2-s Come mostrato nei grafici in figura 7.15, il comportamento cambia radicalmente rispetto alla versione RGB. Il lungo periodo iniziale senza miglioramenti praticamente scompare: il modello inizia a convergere già attorno all'epoca 9–10, con la loss che scende da 0.70 a 0.25 in poche epoche e l'accuracy che raggiunge il 90%+ attorno all'epoca 14. Tuttavia il modello mostra la validation loss più instabile in assoluto: picchi a 1.17 (epoca 16) e 1.63 (epoca 26) che non hanno equivalenti in nessun altro modello. Questi picchi corrispondono a cali temporanei di accuracy (0.69 all'epoca 17, 0.63 all'epoca 26), dopo i quali il modello si riprende. La causa più probabile è che la strategia di riduzione del tasso di apprendimento non sia ben calibrata per la seconda metà del training, e che piccole variazioni nei pesi del transformer amplificano le fluttuazioni sul piccolo validation set. Il risultato finale (≈ 93 –94% di validation accuracy) è comunque nettamente superiore alla versione RGB dello stesso modello.

Considerazioni finali I risultati ottenuti mostrano che i quattro modelli sono in grado di riconoscere le emozioni sulle animazioni MetaHuman con buona affidabilità, indipendentemente dalla modalità di input utilizzata. Su dati RGB i tre modelli CNN raggiungono performance molto simili tra loro ($\approx 94\%$), mentre MViT v2-S fatica ad adattarsi al task nel tempo disponibile, impiegando quasi metà delle epoche prima di iniziare a migliorare in modo significativo. Il passaggio ai dati neuromorfici porta un beneficio generalizzato: tutti i modelli mostrano una maggiore stabilità nelle ultime epoche e, in diversi casi, raggiungono il 100% di validation accuracy (a causa del piccolo dataset utilizzato, semplificando le classificazioni).

Il guadagno più marcato si osserva proprio su MViT v2-S, che su dati event-based converge molto più rapidamente e chiude il divario con gli altri modelli, confermando che la rappresentazione neuromorfica è strutturalmente più adatta a catturare le variazioni temporali sottili presenti nelle sequenze facciali. Un limite comune a tutti i modelli è l'irregolarità della validation loss, che non riflette necessariamente una debolezza del modello ma è in larga parte una conseguenza diretta delle ridotte dimensioni del dataset: con soli 16 video nel validation set, anche un singolo errore di classificazione incide in modo rilevante sulle metriche. Un ampliamento del dataset rappresenta pertanto il passo più importante per ottenere una valutazione più solida e confronti più affidabili tra le architetture. Quest'ultimo magari rafforzato da un aumento delle epoche di training.

8. Conclusioni

In questo lavoro è stata progettata e sviluppata una pipeline automatizzata per la generazione, l'elaborazione e l'utilizzo di dati sintetici finalizzati al riconoscimento delle emozioni. In particolare, sono stati confrontati modelli basati su convoluzioni 2D, approcci ibridi CNN-LSTM, convoluzioni 3D e architetture Transformer, evidenziandone le differenze nella capacità di modellare la componente temporale delle espressioni facciali.

L'approccio proposto si basa sulla generazione di animazioni facciali a partire da segnali audio, successivamente trasformate in sequenze video realistiche e infine convertite in rappresentazioni neuromorfiche. Questo passaggio consente di passare da una rappresentazione tradizionale frame-based a una rappresentazione event-based, maggiormente orientata alla descrizione delle dinamiche temporali del movimento. Tali dati sono stati utilizzati per l'addestramento di diversi modelli di deep learning, consentendo di valutare l'efficacia della pipeline nel contesto del riconoscimento delle emozioni.

I risultati ottenuti mostrano come l'utilizzo di dati sintetici rappresenti una valida alternativa, o complemento, ai dataset reali, permettendo un elevato grado di controllo sulle condizioni di acquisizione e sulle etichette. Inoltre, la conversione in eventi neuromorfici evidenzia le componenti dinamiche delle espressioni facciali, riducendo la ridondanza informativa tipica dei dati RGB e risultando particolarmente adatta all'analisi temporale, contribuendo a una rappresentazione più efficiente del dato.

Inoltre, generando i dati in modo sintetico è possibile simulare in modo abbastanza semplice gli eventi neuromorfici dei video, risolvendo il problema della scarsità dei dataset di espressioni facciali (utili per l'emotion-recognition) di natura neuromorfica. Questo approccio consente un completo controllo sulle variabili ed etichette.

Nonostante i risultati promettenti, il lavoro presenta alcune limitazioni. In particolare, l'utilizzo esclusivo di dati sintetici può introdurre un piccolo gap rispetto ai dati reali, influenzando leggermente la capacità di generalizzazione dei modelli in scenari non controllati. Inoltre, la qualità delle animazioni e delle simulazioni neuromorfiche dipende fortemente dagli strumenti utilizzati e dai parametri scelti.

Come sviluppi futuri, si potrebbe integrare il dataset sintetico con dati reali, al fine di migliorare la robustezza dei modelli. Un'ulteriore direzione riguarda l'impiego di architetture progettate specificamente per dati event-based, al fine di sfruttare appieno le caratteristiche della rappresentazione neuromorfica. Ulteriori miglioramenti potrebbero includere l'ottimizzazione degli iperparametri e l'estensione della pipeline a nuovi scenari applicativi. Infine, sarebbe interessante esplorare l'impiego diretto di sensori neuromorfici reali, al fine di confrontare i risultati con quelli ottenuti tramite simulazione.

In conclusione, il lavoro dimostra la fattibilità e il potenziale di un approccio basato su dati sintetici e simulazioni di eventi neuromorfici per il riconoscimento delle emozioni, evidenziando come la rappresentazione event-based possa costituire un'alternativa efficace ai metodi tradizionali basati su frame, aprendo la strada a possibili sviluppi futuri nell'ambito della visione artificiale e dell'interazione uomo-macchina (HCI).

Glossario

- **Animation Sequence:** serie ordinata di movimenti registrati nel tempo che descrivono come un oggetto o un personaggio deve animarsi. Movimento applicato al *Skeletal Mesh*.
- **Asset:** risorsa digitale utilizzata nel progetto (audio, *mesh*, *Animation Sequence*, ecc...).
- **Bake:** precalcolo e memorizzazione dei dati di un'animazione o di una simulazione, in modo che possano essere riprodotti senza ricalcolare ogni volta.
- **Binding:** associazione tra un oggetto (come un attore, una luce, una camera, un componente o un oggetto Blueprint) e una traccia del Sequencer. Il binding permette al Sequencer di controllare proprietà, animazioni o eventi di quell'oggetto durante la riproduzione della sequenza.
- **Blendshape ARKit:** valore numerico fornito da ARKit (il framework di Apple per realtà aumentata) in cui ogni blendshape ha un nome standard in ARKit (ad esempio *mouthSmileLeft*, *eyeBlinkRight*, *browDownLeft*) e un valore compreso tra 0 e 1, dove 0 indica nessuna attivazione e 1 indica massima attivazione dell'espressione.
- **Blendshape facciale:** tecnica di modellazione e animazione 3D in cui le espressioni del volto sono rappresentate come deformazioni predefinite di una mesh facciale di base. Le espressioni finali vengono ottenute combinando linearmente più blendshape, ciascuno associato a un peso, permettendo la generazione di movimenti ed espressioni facciali realistiche.
- **Control rig:** sistema di controllo in animazione 3D che permette di gestire e animare un modello tramite parametri e controlli, pilotando le sue deformazioni (come blendshape o ossa).
- **Face Skeletal Mesh:** modello 3D della testa o del volto che utilizza una struttura di ossa (*skeleton*) per permettere l'animazione realistica delle espressioni facciali.
- **Keyframe:** fotogramma chiave in un'animazione digitale che definisce un punto specifico di cambiamento o posizione di un oggetto nel tempo. Serve come riferimento per il software per interpolare automaticamente i fotogrammi intermedi tra i keyframe, creando così il movimento fluido.
- **Level Sequence:** *asset* in un motore 3D (come *Unreal Engine*) che contiene una sequenza temporale di eventi, animazioni, suoni e movimenti di camera all'interno di un livello specifico.
- **Livello:** spazio virtuale in cui vengono posizionati e organizzati tutti gli elementi della scena: personaggi, oggetti, luci, telecamere, effetti e suoni.
- **Sequencer:** strumento usato nei motori grafici (come *Unreal Engine*) per creare, gestire e sincronizzare animazioni, effetti visivi e audio lungo una timeline, un po' come un video editor per il mondo 3D.

Bibliografia

- [1] Erlangga Satrio Agung et al. Image-based facial emotion recognition using convolutional neural network. *Scientific Reports*, 2024.
- [2] Lorenzo Berlincioni et al. Neuromorphic event-based facial expression recognition. In *CVPR Workshops*, 2023.
- [3] H. Cao, D. G. Cooper, M. K. Keutmann, R. C. Gur, A. Nenkova, and R. Verma. CREMA-D: Crowd-sourced Emotional Multimodal Actors Dataset. *IEEE Transactions on Affective Computing*, 5(4):377–390, 2014. <https://github.com/CheyneyComputerScience/CREMA-D>.
- [4] Epic Games. Control Rig Scripting Unreal, 2026. https://dev.epicgames.com/documentation/unreal-engine/python-scripting-for-animating-with-control-rig-in-unreal-engine?application_version=5.6.
- [5] Epic Games. Python API, 2026. https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/?application_version=5.6.
- [6] Epic Games. Sequencer Scripting Unreal, 2026. <https://dev.epicgames.com/documentation/unreal-engine/python-scripting-in-sequencer-in-unreal-engine>.
- [7] Daniel Gehrig, Mathias Gehrig, Javier Hidalgo-Carrió, and Davide Scaramuzza. Video to events: Recycling video datasets for event cameras. In *IEEE Conf. Comput. Vis. Pattern Recog. (CVPR)*, June 2020. https://github.com/uzh-rpg/rpg_vid2e.
- [8] Zi-Yu Huang et al. A study on computer vision for facial emotion recognition. *Scientific Reports*, 2023.
- [9] Steven R. Livingstone and Frank A. Russo. The ryerson audio-visual database of emotional speech and song (ravdess), 2018. Published in PLoS ONE, Vol. 13, No. 5, e0196391.
- [10] Gabriele Magrini. Frames2Ev, 2026. <https://github.com/MagriniGabriele/Frames2Ev>.
- [11] Meta. Documentazione PyTorch, 2025. <https://docs.pytorch.org/docs/stable/index.html>.
- [12] NVIDIA. Audio2Face API, 2026. <https://build.nvidia.com/nvidia/audio2face-3d/api>.
- [13] Y. Zhou et al. Computational event-driven vision sensors for in-sensor spiking neural networks. *Nature Electronics*, 2023.

Ringraziamenti

Questo lavoro segna la conclusione del mio percorso triennale, difficile, faticoso e sfidante ma allo stesso tempo appassionante ed in un certo senso divertente.

Inizierei col ringraziare la persona senza la quale non sarebbe stato possibile arrivare fino a questo punto: me stesso. Mi ringrazio per non aver mai preso in considerazione l'idea di mollare, perché la strada, in fondo, è sempre stata una sola.

Questo percorso è stato condiviso anche con i miei compagni di corso, che ringrazio per aver condiviso con me momenti accademici spesso impegnativi e complessi. Un ringraziamento particolare va a *Tommaso Ciccotti*, la prima persona conosciuta all'università, con il quale ho condiviso l'intero percorso di studio, dal primo giorno. Ti ringrazio anche per aver progettato e sviluppato insieme a me il progetto di Ingegneria del Software, di cui vado tuttora molto fiero. Ti ringrazio inoltre per aver sempre accolto, supportato e talvolta sopportato le mie idee ambiziose e a volte eccessive, ma sempre orientate a raggiungere un risultato il più possibile completo e di cui poter andare fieri anche in futuro.

Desidero ringraziare, in particolare per il supporto ricevuto durante questo lavoro di tesi, il mio relatore, il *Prof. Pietro Pala*, e i correlatori il *Dott. Gabriele Magrini* e il *Prof. Federico Becattini*. Sono lieto di avervi scelto per la supervisione di questo lavoro, e vi ringrazio per la disponibilità, la gentilezza e il costante supporto, soprattutto nei momenti più critici dello sviluppo.

Ringrazio i miei amici.

Un ringraziamento molto speciale va infine alla mia famiglia, alla quale dedico questo lavoro, che mi ha sempre sostenuto fin dal primo giorno, senza mai smettere di credere in me. Avete sempre appoggiato le mie idee e le mie scelte, anche quando non erano pienamente condivise, mettendo sempre il mio bene al primo posto e cercando di fare ciò che fosse meglio per me, anche quando questo poteva entrare in conflitto con le vostre convinzioni o aspettative.

Ti amo Mamma.

Ti amo Virgi.

Ti amo Babbo.

Ma questo è solo l'inizio...